

۳ حل مسأله از طریق جستجو

در این فصل می‌بینیم که یک کارگزار، هنگامی که نمی‌تواند تنها با انجام یک اقدام به هدف برسد، چگونه می‌تواند یک رشته اقدامات را برای این کار بیابد.

$E \Rightarrow F$
Search
جستجو
Reflex agent
کارگزار واکنشی
Environment
محیط
Problem - solving agent
کارگزار حل مسأله
Desired state
حالت مطلوب
Elements
عناصر
Asymptotic Complexity
پیچیدگی
مجانسی
Solution
راه حل
General purpose
همه‌منظوره
Uninformed
ناآگاه
Informed
آگاه

ساده‌ترین کارگزارهایی که در فصل ۲ مطرح شدند، کارگزارهای واکنشی بودند که اقداماتشان را بر اساس یک نگاشت مستقیم از حالتها به اقدامات قرار می‌دهند. این نوع کارگزارها در محیطهایی که در آنها ذخیره‌سازی این نگاشت بیش از حد بزرگ است و یادگیری آن بیش از حد طول می‌کشد، نمی‌توانند به خوبی عمل کنند. درحالی که کارگزارهای مبتنی بر هدف می‌توانند با در نظر گرفتن اقدامات آینده و ارزش پیش‌آمدهای آنها، به موفقیت دست یابند.

در این فصل، یک نوع کارگزار مبتنی بر هدف به نام کارگزار حل مسأله را معرفی می‌کنیم. کارگزار حل مسأله با پیدا کردن رشته‌هایی از اقدامات که به حالت‌های مطلوب منجر می‌شوند، تصمیم می‌گیرد که چه عملی انجام دهد. ابتدا با تعریف دقیق عناصری که یک "مسأله" را تشکیل می‌دهند و نیز "راه حل" آن بحث را شروع می‌کنیم و چندین مثال برای روشن ساختن این تعاریف مطرح می‌کنیم. سپس، چند الگوریتم جستجوی همه‌منظوره را توضیح می‌دهیم که برای حل این مسائل می‌توانند استفاده شوند و مزایای این الگوریتمها را مقایسه می‌کنیم. این الگوریتمها ناآگاه هستند، به این معنی که درباره مسأله هیچ اطلاعاتی به جز تعریف آن، به آنها داده نشده است. فصل ۴ به الگوریتمهای جستجوی آگاهانه می‌پردازد که در آن درباره روشهایی صحبت می‌شود که محلهای راه حل را آدرس‌دهی می‌کنند.

این فصل مفاهیمی از تحلیل الگوریتمها را به کار می‌برد. خوانندگانی که با مفاهیم پیچیدگی مجانبی (که به شکل $O()$ نوشته می‌شود) و NP -کامل آشنا نیستند، باید به پیوست الف مراجعه کنند.

۱-۳ کارگزارهای حل مسأله

کارگزارهای هوشمند باید مقیاس کارایی خود را به حداکثر برسانند. همان‌گونه که در فصل ۲ گفتیم، گاهی دست‌یابی به کارگزاری که بتواند هدفی را اتخاذ و آن را برآورده کند، امری ساده است. ابتدا بررسی می‌کنیم که چرا و چگونه یک کارگزار ممکن است چنین کاری را انجام دهد.

یک کارگزار را در نظر بگیرید که در شهر Arad در رومانی تعطیلات خود را می‌گذراند. مقایسه کارایی این کارگزار، فاکتورهای متعددی دارد: برنزه شدن، یادگیری زبان رومانیایی، لذت بردن از مناظر، مدت بردن از مقریحات شبانه، حفظ سلامتی و غیره. مسأله تصمیم‌گیری، مسأله‌ای است که شامل بدیهه‌های زیاد و مطالعه دقیق کتابچه‌های راهنماست. حال، فرض کنید که کارگزار یک بلیط غیر قابل استرداد برای پرواز به Bucharest برای روز بعد دارد. در این حالت، معقول است که کارگزار هدف رسیدن به Bucharest را اتخاذ کند. مجموعه اقداماتی که سر موقع، وی را به Bucharest می‌رساند را می‌توان بدون نیاز به رسیدگی بیشتر رد کرد و مسأله تصمیم‌گیری کارگزار بسیار ساده می‌شود. اهداف با محدود کردن مقاصد که کارگزار برای رسیدن به آنها تلاش می‌کند، به سازمندی رفتاری کارگزار کمک می‌کنند. اولین گام در حل مسأله تدوین هدف بر اساس حالت فعلی و مقیاس کارایی کارگزار می‌باشد.

ما هدف را به صورت مجموعه‌ای از حالت‌های دنیا در نظر می‌گیریم (دقیقاً همان حالتی که در آنها هدف برآورده شده است). وظیفه کارگزار، فهمیدن این مطلب است که کدام رشته از اقدامات وی را به یک حالت هدف می‌رساند. کارگزار، پیش از انجام این کار باید تصمیم بگیرد که چه اقدامات و حالت‌هایی را مد نظر قرار دهد. اگر قرار بود اقداماتی در سطح "پای چپ را یک سانتی متر جلو ببر" یا "فرمان را یک درجه به چپ بچرخان" را در نظر بگیرد، احتمالاً کارگزار هرگز راه خروج از بارکینگ را پیدا نمی‌کرد. چه برسد به رفتن به Bucharest. در این سطح از جزئیات، عدم قطعیت بسیار زیادی در دنیا وجود دارد و یک راه حل، شامل مراحل بسیار زیادی خواهد بود. تدوین مسأله، فرایند تصمیم‌گیری در مورد اقدامات و حالتها بر اساس یک هدف خاص است. درباره این فرایند بعداً بیشتر بحث خواهیم کرد. در حال حاضر، فرض می‌کنیم که کارگزار اقداماتی در سطح رانندگی از یک شهر اصلی به شهر اصلی دیگر را در نظر خواهد گرفت. حالت‌هایی که در نظر می‌گیرد نیز متناسب با اقامت در یک شهر خاص خواهد بود.^۱

اکنون کارگزار ما هدف رانندگی به Bucharest را اتخاذ کرده است و در این باره فکر می‌کند که از Arad به کجا برود. سه جاده از Arad خارج می‌شود، یکی به سمت Sibiu یکی به سمت Timisoara و یکی به زربند. هیچ یک از این جاده‌ها ما را به هدف نمی‌رساند. بنابراین، چنانچه کارگزار به جغرافیای رومانی کاملاً آشنا نباشد، نمی‌داند که چه جاده‌ای را انتخاب کند. به عبارت دیگر، کارگزار نخواهد دانست که کدام یک از اقدامات ممکن بهترین است، زیرا دقیقاً نمی‌داند که از انجام هر اقدام، چه حالتی حاصل می‌شود. اگر کارگزار هیچ اطلاعات اضافی نداشته باشد، گیر افتاده است. بهترین کاری که می‌تواند انجام دهد، انتخاب یکی از اقدامات به صورت تصادفی است.

حال فرض کنید که کارگزار دارای یک نقشه از کشور رومانی، به صورت کاغذی یا در حافظه‌اش باشد. هدف نقشه، فراهم کردن اطلاعات درباره حالت‌هایی است که ممکن است کارگزار به آنها برسد و نیز اقداماتی که می‌تواند انجام دهد. کارگزار با استفاده از این اطلاعات

هدف	Target
موقعیت	Location
موقعیت فعلی	Current location
موقعیت مورد نیاز	Required location
هدف	Goal
روز ۱۹۹۲	To 1992
روز تمرین	Training day
مسئله	Problem
تصمیم‌گیری	Decision making
سازماندهی	Organizing
تدوین هدف	Goal formulation
فرمان	Steering wheel
عدم قطعیت	Uncertainty
تدوین مسأله	Problem formulation
آراد	Arad
سبوی	Sibiu
تیمی سوار	Timisoara
زربند	Zerind
رومانی	Romania
نقشه	Map

^۱ توجه کنید که هر یک از این "حالت‌ها" مناظر یا مجموعه بزرگی از حالت‌های دنیا است. زیرا هر حالت دنیای واقعی، تمام جنبه‌های واقعیت را مشخص می‌کند. متمایز دانستن حالات حل مسأله و حالت‌های دنیا اهمیت زیادی دارد. ما فرض کرده‌ایم که اکثر خوانندگان در کشورهای مختلف هستند و در نتیجه به آسانی مانند کارگزار، گیج می‌شوند از خوانندگان رومانیایی که در این مثال آموزشی می‌بهره می‌مانند عذرخواهی می‌کنیم.

جستجو = فرآیند پیدا کردن راهی مناسب از حالت‌های ممکن

۷۳

فصل ۳- حل مسأله از طریق جستجو

ف	E
Sequence	توالی
Search	جستجو
Input	ورودی
Action sequence	رشته اقدام
Execution phase	مرحله اجرا
Search procedure	روال جستجو
To formulate	تدوین کردن
Simple problem-solving agent	کارگزار ساده
Percepts	حالت مسأله
Known values	ادراکات
To call	مقادیر معلوم
Complete	فراخوانی کردن
Process	کامل
Function	فرآیند
Path	تابع
Complete	مسیر
	کامل

مراحل متوالی یک سفر فرضی از طریق یکی از این سه شهر را مورد مطالعه قرار می‌دهد تا مسیری را بیابد که نهایتاً به Bucharest برسد. زمانی که روی نقشه مسیری را از Arad به Bucharest پیدا کند، می‌تواند با انجام اقدام رانندگی در قسمتهای مسیر سفر به هدفش برسد. در کل، کارگزاری با چندین گزینه از مقادیر نامعلوم، بدین شکل می‌تواند تصمیم بگیرد که چه کاری انجام دهد: در ابتدا، توالیهای مختلف ممکن از اقدامات را که به حالت‌هایی با مقادیری معلوم منجر می‌شوند، بررسی کند و سپس بهترین آنها را انتخاب کند.

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) returns an action

inputs: *percept*, a percept ادراکات

static *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null هدف

problem, a problem formulation

state ← UPDATE-STATE(*state*, *percept*)

if *seq* is empty then do

goal ← FORMULATE-GOAL(*state*)

problem ← FORMULATE-PROBLEM(*state*, *goal*)

seq ← SEARCH(*problem*)

action ← FIRST(*seq*)

seq ← REST(*seq*)

return *action*

شکل ۳-۱ یک کارگزار ساده حل مسأله، ابتدا یک هدف و یک مسأله را تدوین می‌کند، به دنبال یک رشته اقدام می‌گردد که مسأله را حل خواهد کرد و سپس اقدامات را یکی یکی اجرا می‌کند. هنگامی که کامل شد، یک هدف دیگر را تدوین می‌کند و دوباره از نو همه این کارها را انجام می‌دهد. دقت داشته باشید که کارگزار هنگام اجرای رشته اقدام، ادراکاتش را نادیده می‌گیرد و فرض می‌کند راه‌حلی که یافته است، همیشه درست عمل می‌کند.

این فرآیند را که به دنبال چنین توالی می‌گردد، جستجو می‌نامیم. الگوریتم جستجو، یک مسأله را به عنوان ورودی می‌پذیرد و یک راه‌حل را به شکل یک رشته اقدام برمی‌گرداند. به محض این که یک راه‌حل پیدا شد، می‌توان اقداماتی را که پیشنهاد کرده است، انجام داد. این مرحله را مرحله اجرا می‌نامیم. بنابراین، همان‌طور که شکل ۳-۱ نمایش می‌دهد، کارگزار به صورت "تدوین، جستجو، اجرا" طراحی می‌شود. پس از تدوین یک هدف و یک مسأله برای حل، کارگزار یک روال جستجو را برای حل آن فراخوانی می‌کند. سپس، از راه‌حل برای انتخاب اقدام‌هایش استفاده می‌کند. راه حل مشخص می‌کند که در مرحله بعدی چه اقدامی انجام دهد. پس از انجام آن اقدام توسط کارگزار، آن مرحله از رشته اقدامات حذف می‌شود. هنگامی که راه‌حل اجرا گردید، کارگزار هدف جدیدی را تدوین می‌کند.

ابتدا، فرآیند تدوین مسأله را توضیح می‌دهیم و سپس در بقیه فصل به الگوریتم‌های

مختلفی که برای تابع *SEARCH* می‌توان استفاده کرد، می‌پردازیم. در این فصل، نحوه عملکرد توابع *UPDATE-STATE* و *FORMULATE-GOAL* را مورد بحث قرار نمی‌دهیم.

پیش از این که وارد جزئیات شویم، کمی مکث می‌کنیم تا بینیم کارگزارهای حل مسأله در بحث کارگزارها و محیطهایی که در فصل ۲ مطرح شد، در کجا قرار می‌گیرند. در طراحی کارگزار شکل ۱-۳ فرض می‌شود که محیط ایستا است، زیرا تدوین و حل مسأله، بدون توجه به تغییراتی که ممکن است در محیط اتفاق بیافتد انجام می‌شود. همچنین، در طراحی کارگزار فرض می‌شود که حالت ابتدایی شناخته شده است، زیرا اگر محیط رؤیت‌پذیر باشد، کار بسیار ساده‌تر می‌شود. ایده برشماری "رشته اقدامات مختلف" فرض می‌کند که محیط را می‌توان به صورت گسسته در نظر گرفت. بالاخره و از همه مهمتر، طراحی کارگزار فرض می‌کند که محیط معین است. راه‌حلهای مسائل، رشته‌های ساده‌ای از اقدامات هستند. بنابراین، نمی‌توانند از عهده هیچ رویداد غیرمنتظره‌ای برآیند. علاوه بر این، راه‌حلهای بدون توجه به ادراکات اجرا می‌شوند! کارگزاری که طرحش را با چشمان بسته اجرا می‌کند، باید کاملاً از آنچه می‌گذرد مطمئن باشد. (نظریه پردازان پایش، این سامانه را حلقه باز می‌نامند، زیرا بی‌توجهی به ادراکات، حلقه بین کارگزار و محیط را از بین می‌برد.) تمامی این فرضها به این معنی است که ما با ساده‌ترین نوع محیطها سرو کار داریم. یکی از دلایلی که این فصل در ابتدای کتاب آمده است، همین است. بخش ۳-۶ نگاه مختصری به این حالت می‌اندازد که اگر فرضیات رؤیت‌پذیری و معین بودن را برداریم چه اتفاقی خواهد افتاد. فصلهای ۱۲ و ۱۷ این مسأله را با عمق بیشتری بررسی می‌کنند.

مسائل خوش تعریف و راه‌حلهای

یک مسأله، به صورت رسمی با چهار مؤلفه تعریف می‌شود:

- **حالت ابتدایی** که در آن حالت کارگزار شروع به کار می‌کند. برای مثال، حالت ابتدایی کارگزار ما در رومانی، ممکن است به شکل *In(Arad)* توصیف شود.
- **تعریف اقدامات ممکن** که در دسترس کارگزار هستند. رایج‌ترین تدوین^۳ از یک تابع پسین استفاده می‌کند. با فرض حالت مشخص x *SUCCESSOR-FN* یک مجموعه از زوجهای مرتب (پسین، اقدام) برمی‌گرداند که هر اقدام یکی از اقدامهای مجاز در حالت x می‌باشد و هر پسین، حالتی است که از حالت x با اعمال آن اقدام می‌توان به آن رسید. برای مثال، از حالت *In(Arad)* تابع پسین مسأله رومانی چنین چیزی برمی‌گرداند:

$\{ \langle \text{Go}(\text{Sibiu}), \text{In}(\text{Sibiu}) \rangle, \langle \text{Go}(\text{Timisoara}), \text{In}(\text{Timisoara}) \rangle, \langle \text{Go}(\text{Zerind}), \text{In}(\text{Zerind}) \rangle \}$

حالت ابتدایی و تابع پسین، همراه یکدیگر به طور ضمنی، فضای حالت مسأله را تعریف می‌کنند (مجموعه تمامی حالتهایی که از حالت ابتدایی می‌توان به آنها رسید). فضای حالت، گرافی را تشکیل می‌دهد که گرههای آن حالتها هستند و کمانهای بین گرهها، اقدامات. (نقشه رومانی که در شکل ۲-۳ نشان داده شده است را می‌توانیم یک گراف فضای حالت فرض کنیم

E ⇔ F

To pause

مکث کردن

Static

ایستا

Observable

رؤیت‌پذیر

Enumerating

برشماری

Discrete

گسسته

Deterministic

معین

Open-loop

حلقه باز

Well-defined

خوش تعریف

Formal

رسمی

Component

مؤلفه

Initial state

حالت ابتدایی

Possible

actions

اقدامات ممکن

Formulation

تدوین

Ordered pairs

زوجهای مرتب

Successor

پسین

Arc

کمان

Operator

عملگر

Map

نقشه

Function

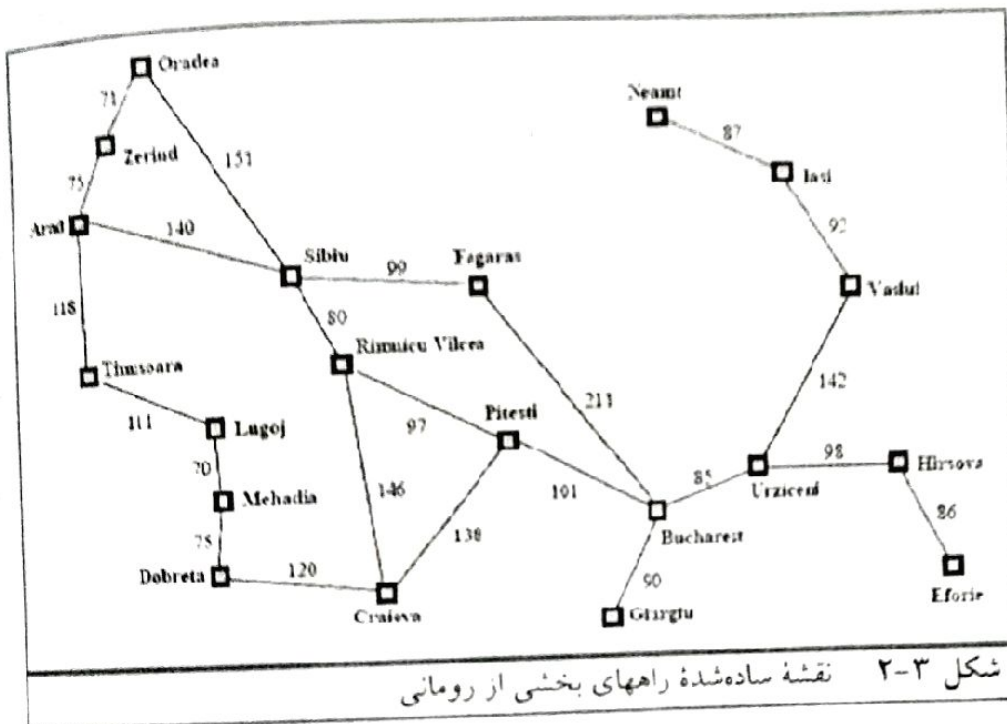
تابع

^۳ یک تدوین جایگزین، از یک مجموعه عملگر استفاده می‌کند که می‌توان به یک حالت اعمال کرد تا پسینها را تولید کند.

ف $E \Rightarrow$	اگر هر جاده را متناظر با دو اقدام. هر کدام در یک جهت بدانیم. یک مسیر در فضای حالت،
Goal test	یک توالی از حالت‌هایی است که توسط رشته اقدامات به هم متصل شده‌اند.
آزمون هدف	
Goal state	• آزمون هدف، تعیین می‌کند که یک حالت خاص، حالت هدف است یا نه. گاهی اوقات، یک
حالت هدف	مجموعه صریح از حالت‌های هدف ممکن وجود دارد و آزمون فقط بررسی می‌کند که آیا
Explicit	حالت داده شده یکی از آنها هست یا نه. هدف کارگزار در رومانی، مجموعه تک عضوی $\{In$
صریح	(Bucharest) است. گاهی اوقات هدف به جای یک مجموعه صریح از حالت‌های
Property	مشخص با استفاده از یک ویژگی چکیده توصیف می‌شود. برای مثال، در بازی شطرنج،
ویژگی	هدف رسیدن به حالتی به نام "کیش و مات" است که در آن مهره شاه حریف، مورد حمله
Chess	قرار می‌گیرد و راه گریز هم ندارد.
شطرنج	• یک تابع هزینه مسیر که به هر مسیر، یک هزینه عددی تخصیص می‌دهد. کارگزار حل مسأله
Checkmate	تابع هزینه‌ای را انتخاب می‌کند که مقیاس کارآیی خودش را منعکس کند. برای کارگزاری که
کیش و مات	برای رسیدن به Bucharest تلاش می‌کند، زمان موضوع اصلی و محوری می‌باشد. بنابراین،
Path cost function	هزینه مسیر می‌تواند طول مسیر بر حسب کیلومتر باشد. در این فصل فرض می‌کنیم که هزینه
تابع هزینه مسیر	مسیر، می‌تواند به عنوان مجموع هزینه‌های اقدامات مجزا در طول مسیر توصیف شود. هزینه
Step cost	گام برای اجرای اقدام a که از حالت x به حالت y می‌برد، به شکل $C(x, a, y)$ نمایش داده
هزینه گام	می‌شود. هزینه‌های گام در مسأله رومانی، در شکل ۳-۲ به عنوان مسافت مسیر نشان داده
Input	شده‌اند. فرض می‌کنیم که هزینه گام منفی نیست!
ورودی	
Problem formulation	عناصری که ذکر شد، یک مسأله را تعریف می‌کنند و می‌توان تمامی آنها را در یک ساختار داده
تدوین مسأله	منفرد گردآوری کرد و به عنوان ورودی به الگوریتم حل مسأله ارائه کرد. راه حل یک مسأله،
Real world	مسیری از حالت ابتدایی تا حالت هدف است. کیفیت راه حل را با تابع هزینه مسیر اندازه‌گیری
دنیای واقعی	می‌کنند و یک راه حل بهینه کمترین هزینه مسیر را در بین تمامی راه‌حلها دارد.
Police officer	
افسر پلیس	
Station	
ایستگاه	
Road	
جاده	
Abstraction	تدوین مسأله رسیدن به Bucharest را با استفاده از حالت ابتدایی، تابع پسین،
چکیدگی، تجرید	آزمون هدف و هزینه مسیر ارائه کردیم. این روش تدوین معقول به نظر می‌رسد. با وجود این،
Abstract	بسیاری از جنبه‌های دنیای واقعی از قلم می‌افتد. توصیف ساده حالتی را که انتخاب کرده‌ایم،
چکیده، مجرد	یعنی $In (Arad)$ ، با یک سفر واقعی در عرض یک کشور مقایسه کنید که در آنجا حالت دنیا،
Fuel consumption	چیزهای زیادی را شامل می‌شود: همراهان در سفر، آنچه رادیو پخش می‌کند، منظره بیرون
مصرف سوخت	پنجره، حضور افسر پلیس در نزدیکی، فاصله تا ایستگاه بعدی، حالت جاده، آب و هوا و غیره.
Pollution	تمامی این مسائل خارج از توصیف حالت قرار گرفته‌اند، زیرا نسبت به مسأله یافتن مسیری به
آلودگی	Bucharest بی‌ربط هستند. فرآیند حذف جزئیات از یک بازنمایی را چکیده کردن می‌نامند.
	علاوه بر حالتها، باید اقدامات را هم چکیده کنیم. یک اقدام رانندگی، اثرات زیادی دارد.
	همچنین تغییر محل خودرو و سرنشینان آن، زمان می‌برد، سوخت مصرف می‌کند، آلودگی

۴ جنبه‌های مختلف هزینه‌های منفی را در تمرین ۳-۱۷ بررسی می‌کنیم.

می‌کند و کارگزار را تغییر می‌دهد (می‌گویند سفر آدمی را پنخته می‌کند). ولی در تداومش که ما انجام دادیم، فقط تغییر محل را به حساب آوردیم. اقدامات زیادی نیز هستند که گلاً از قلم می‌اندازیم: روشن کردن رادیو، از پنجره بیرون را نگاه کردن، پواش کردن به خاطر افسردگی غیره. البته، اقداماتی در سطح "فرمان را به اندازه سه درجه به سمت چپ بچرخان" را هم ذکر نمی‌کنیم.



شکل ۲-۳ نقشه ساده‌شده راههای بخشی از رومانی

آیا می‌توان تعریف دقیق‌تری درباره سطح مناسب چکیدگی ارائه کرد؟ حالتها و اقدامات چکیده‌ای که انتخاب کردیم را متناظر با مجموعه‌های بزرگی از حالات پر از جزئیات دنیا و رشته اقداماتی پر از جزئیات در نظر بگیرید. حال، یک راه‌حل برای این مسئله چکیده در نظر بگیرید. برای مثال، مسیری از Arad به Sibiu، Rimnicu Vilcea، Pitesti و Bucharest این راه‌حل چکیده متناظر با مسیرهای زیادی با جزئیات بیشتر است. مثلاً می‌توانیم از Sibiu تا Rimnicu Vilcea رادیو را روشن بگذاریم و سپس برای بقیه مسیر رادیو را خاموش کنیم. چکیدگی در صورتی معتبر است که بتوانیم هر راه‌حل چکیده را به یک راه‌حل مفصل‌تر در دنیای واقعی گسترش دهیم؛ شرط کافی این است که برای هر حالت مشروح که "در Arad" است، مسیر مشروحی به حالت "در Sibiu" وجود داشته باشد و مشابه این چکیدگی را سودمند می‌دانیم اگر اجرای هر یک از اقدامات راه‌حل از مسئله اصلی ساده‌تر باشد؛ در این مورد خاص، باید آن قدر ساده باشند که بدون جستجو یا برنامه‌ریزی بیشتر توسط یک کارگزار رانده معمولی، قابل انجام باشند. در نتیجه، انتخاب یک چکیدگی خوب، شامل حذف جزئیات تا حد امکان می‌شود (به شرط حفظ اعتبار) و مطمئن باشد که اجرای اقدامات چکیده ساده‌تر هستند. اگر توانایی چکیده کردن نبود، کارگزارهای هوشمند کاملاً در دنیای واقعی غرق می‌شدند.

E	ف
To omit	از قلم انداختن
To slow down	پواش کردن
Degree	درجه
Precise	دقیق
Appropriate	مناسب
Set	مجموعه
Arad	آراد
Sibiu	سیبوی
Rimnicu Vilcea	ریمنیکو ویلچا
Pitesti	پیتستی
Bucharest	بخارست
Detailed	مشروح
Details	جزئیات
Useful	سودمند
Valid	معتبر
Sufficient condition	شرط کافی
Original problem	مسئله اصلی

۲-۳ مسائل نمونه

ف $\Rightarrow$ E

Example problems

مسائل نمونه

Toy problems

مسائل

دست گرمی

Concise

موجز

Comparison

مقایسه

To formulate

تدوین کردن

Initial state

حالت ابتدایی

State space

فضای حالت

Arc

کمان

Successor function

تابع پسین

Legal

مجاز

Goal test

آزمون هدف

Square

مربع

Path cost

هزینه مسیر

Action

اقدام

State space

فضای حالت

Step

گام، مرحله

To result

نتیجه شدن

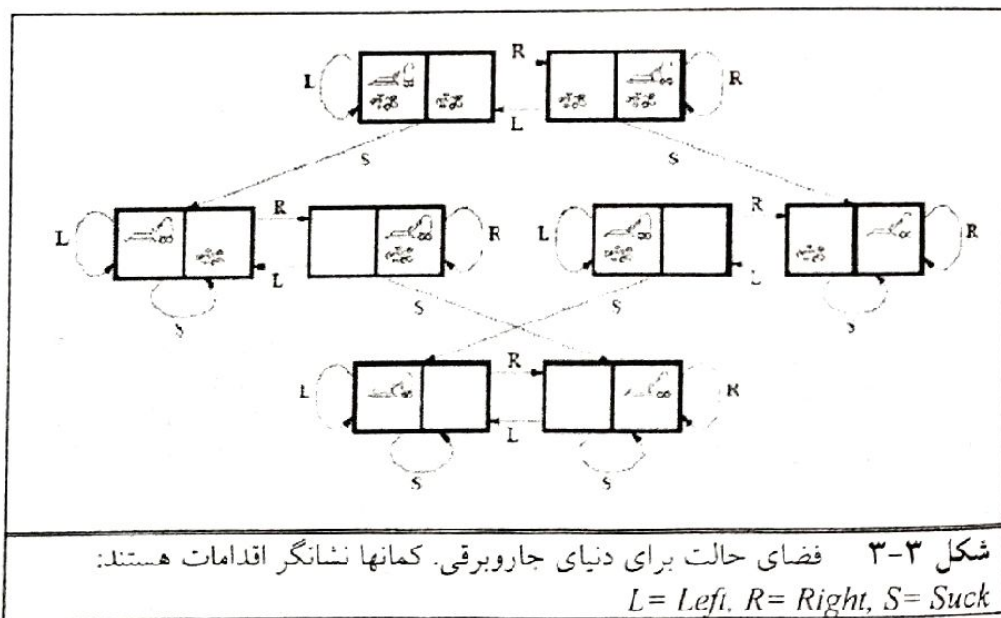
رویکرد حل مسأله در مجموعه وسیعی از محیطهای کار استفاده می شود. در اینجا بعضی از شناخته شده ترین این رویکردها را با در نظر گرفتن تفاوت بین مسائل دست گرمی و دنیای واقعی فهرست می کنیم. هدف یک مسأله دست گرمی نمایش یا تمرین روشهای مختلف حل مسأله است. می توان یک تعریف دقیق و موجز برای آن ارائه کرد. این بدان معنی است که پژوهشگران مختلف می توانند از آنها، برای مقایسه کارایی الگوریتمها استفاده کنند. یک مسأله دنیای واقعی مسأله ای است که راه حل های آن برای مردم واقعاً اهمیت دارد. برای این گونه مسائل معمولاً یک تعریف واحد که همگی روی آن توافق داشته باشند وجود ندارد، ولی ما سعی می کنیم ویژگی عمومی تدوین آنها را ارائه کنیم.

مسائل دست گرمی

اولین مثالی که بررسی می کنیم، دنیای جاروبرقی است که در فصل ۲ برای اولین بار معرفی شد. (شکل ۲-۲ را نگاه کنید) این دنیا را می توان را به شکل زیر به عنوان یک مسأله تدوین کرد.

◊ **حالتها:** کارگزار در یکی از دو محل قرار دارد که هر از این دو محل ممکن است آشغال داشته باشد یا نداشته باشد. بنابراین، $2 \times 2^2 = 8$ حالت ممکن دنیا وجود دارد.

◊ **حالت شروع:** هر حالتی ممکن است حالت ابتدایی در نظر گرفته شود.



◊ **تابع پسین:** این تابع، حالت های مجاز که از انجام هر یک، سه اقدام (Left, Right, Suck) نتیجه می شوند را تولید می کند. کل فضای حالت در شکل ۳-۳ نشان داده شده است.

◊ **آزمون هدف:** این آزمون تمیز بودن تمامی مربعها را بررسی می کند.

◊ **هزینه مسیر:** هر گام ۱ واحد هزینه دارد. بنابراین، هزینه مسیر، تعداد گامهای مسیر

است.

در مقایسه با دنیای واقعی، این مسأله دست‌گرمی، دارای محلهای گسسته، اشغال گسسته و تمیز کردن قابل اعتماد است و یک بار که تمیز شد، دیگر کثیف نمی‌شود. (در بخش ۳-۶ این فرضیات را حذف می‌کنیم.) یک مسأله اساسی توجه به این مطلب است که حالت هم بر اساس محل کارگزار و هم محلهای اشغال تعیین می‌شود. محیط بزرگ‌تری که n محل داشته باشد، n^2 حالت خواهد داشت.

پازل هشت‌تایی که یک نمونه آن در شکل ۳-۴ نشان داده شده است، از یک تخته 3×3 با ۸ کاشی شماره‌دار و یک فضای خالی تشکیل می‌شود. هر کاشی که مجاور فضای خالی قرار داشته باشد، می‌تواند به سمت فضای خالی بلغزد. هدف، رسیدن به یک حالت هدف مشخص، مانند حالتی که در سمت راست شکل نشان داده شده است، می‌باشد. تدوین استاندارد به شکل زیر است:

◊ **حالتها:** یک حالت، محل هر یک از هشت کاشی و فضای خالی را در نه مربع مشخص می‌کند.

◊ **حالت ابتدایی:** هر حالتی را می‌توان به عنوان حالت ابتدایی در نظر گرفت. توجه داشته باشید که می‌توان دقیقاً از نیمی از حالت‌های ابتدایی ممکن، به هر هدف مشخصی رسید. (تمرین ۳-۴)

◊ **تابع پسین:** این تابع، حالت‌های مجاز که از انجام چهار اقدام (حرکت فضای خالی به چپ، راست، بالا یا پایین) نتیجه می‌شود را تولید می‌کند.

◊ **آزمون هدف:** بررسی می‌کند که حالت با حالت هدف که در شکل ۳-۴ نشان داده شده است، مطابقت دارد یا نه. (می‌توان شکل‌های دیگری را به عنوان هدف در نظر گرفت.)

◊ **هزینه مسیر:** هر گام ۱ واحد هزینه دارد. بنابراین، هزینه مسیر، تعداد گام‌های آن مسیر می‌باشد.

E ⇔ F

Discrete

گسسته

Reliable

قابل اعتماد

8-puzzle

پازل هشت‌تایی

Board

تخته

Tile

کاشی

Numbered

شماره‌دار

Adjacent

مجاور

Blank space

فضای خالی

To slide

لغزیدن

Standard

formulation

تدوین استاندارد

Initial state

حالت ابتدایی

Successor

function

تابع پسین

Goal state

آزمون هدف

Path cost

هزینه مسیر

Legal

مجاز

Step

گام

Instance

نمونه

To reach

رسیدن

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

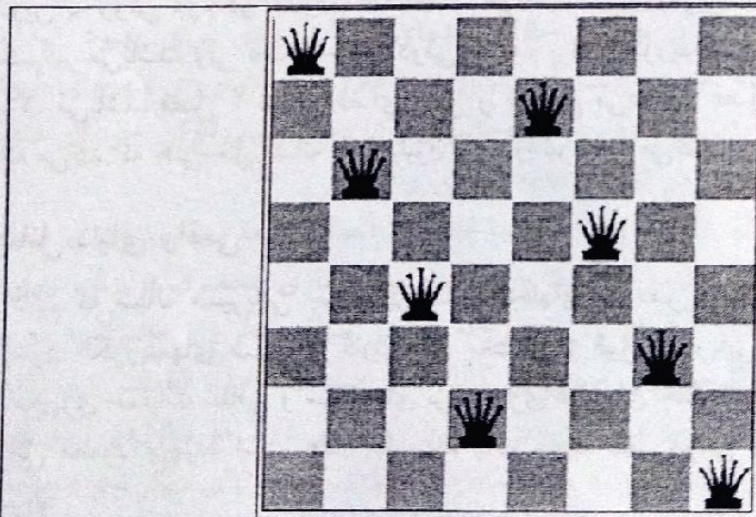
شکل ۳-۴ یک نمونه از پازل هشت‌تایی

E	ف
Abstraction	چکیدگی، تجرید
Sliding-block puzzles	پازل‌های بلوک لغزنده
Test problems	مسائل آزمایشی
Worst case	بدترین حالت
8-puzzle	پازل هشت‌تایی
Optimal	بهینه
Current	فعلی
8-queens problem	مسأله ۸ وزیر
Row	ردیف
Column	ستون
Diagonal	قطر
Incremental formulation	تدوین افزایشی
Operator	عملگر
Description	توصیف
Search algorithm	الگوریتم جستجو
Special-purpose	خاص منظوره

چه چکیدگی‌هایی را در اینجا منظور کرده‌ایم؟ اقدام به حالت‌های شروع و پایان چکیده شده است و محل‌های میانی که بلوک در حال لغزیدن است نادیده گرفته شده‌اند. اقداماتی مانند تکان دادن تخته وقتی که قطعات گیر کرده‌اند، یا درآوردن قطعات با استفاده از چاقو و جاگذاری سر جایشان را حذف کرده‌ایم. بدون در نظر گرفتن جزئیات عملکرد دستی و فیزیکی، تنها تعریف قوانین جدول باقی می‌ماند.

مسأله پازل هشت‌تایی به خانواده پازل‌های بلوک لغزنده تعلق دارد که اغلب به عنوان مسائل آزمایشی برای الگوریتم‌های جدید جستجو در هوش مصنوعی استفاده می‌شوند. این دسته از مسائل را NP -کامل می‌دانند. بنابراین، کسی انتظار ندارد که روش‌هایی بیابد که در بدترین حالت به طور قابل ملاحظه‌ای بهتر از الگوریتم‌های جستجویی که در این فصل و فصل بعدی توضیح می‌دهیم باشد. پازل هشت‌تایی $9!/2 = 181440$ حالت ممکن دارد و به‌سادگی قابل حل است. پازل پانزده‌تایی (روی یک تخته 4×4) حدود 1.3 تریلیون حالت دارد و نمونه‌های تصادفی، با بهینه‌ترین الگوریتم‌های جستجو در چند میلی‌ثانیه قابل حل هستند. پازل ۲۴ تایی (روی تخته 5×5) در حدود 10^{25} حالت دارد و حل بهینه نمونه‌های تصادفی، توسط ماشین‌ها و الگوریتم‌های فعلی هنوز بسیار مشکل می‌باشد.

هدف مسأله ۸ وزیر، چیدن ۸ مهره وزیر روی صفحه شطرنج است به گونه‌ای که هیچ وزیری به وزیر دیگر حمله نکند. (وزیر به هر مهره‌ای که در همان ردیف، ستون یا قطر قرار داشته باشد، حمله می‌کند.) شکل ۳-۵ تلاشی برای یافتن راه‌حل را نشان می‌دهد که به شکست انجامیده است. وزیری که در آخرین ستون سمت راست قرار دارد، توسط وزیر اولین ستون سمت چپ مورد حمله قرار گرفته است.



شکل ۳-۵ یک راه‌حل نصفه‌نیمه برای مسأله ۸ وزیر (راه‌حل کامل به عنوان تمرین باقی می‌ماند).

با وجود این که الگوریتم‌های خاص منظوره‌ای برای این مسأله و کل خانواده n وزیر وجود دارد، این مسأله به عنوان یک مسأله آزمایشی جذاب برای الگوریتم‌های جستجو باقی مانده است. برای این مسأله دو نوع تدوین وجود دارد. **تدوین افزایشی** شامل عملگرهایی است که توصیف

حالت را که از یک حالت خالی شروع می‌شود، بزرگ‌تر می‌کنند. در مورد مسأله هشت وزیر، به این معنی است که هر اقدام یک وزیر به حالت اضافه می‌کند. تدوین حالت کامل، با ۸ وزیر روی تخته شروع می‌کند و آنها را به اطراف حرکت می‌دهد. در هر دو حالت، هزینه مسیر اهمیت ندارد زیرا تنها حالت نهایی مهم است. اولین تدوین افزایشی که ممکن است به ذهن برسد به شرح زیر می‌باشد:

- ◊ حالتها: هر چیدمانی از صفر تا ۸ وزیر روی صفحه یک حالت است.
- ◊ حالت ابتدایی: هیچ وزیری روی صفحه نباشد.
- ◊ تابع پسین: افزودن یک وزیر به هر خانه خالی.
- ◊ آزمون هدف: ۸ وزیر روی صفحه قرار دارند که هیچ یک به دیگری حمله نمی‌کند.

در این تدوین، $3 \times 10^{14} \approx 57 \times 64 \times 64 \times 0.0001$ توالی ممکن برای بررسی وجود دارد. یک تدوین بهتر، از قرار گرفتن وزیر در خانه‌ای که تحت حمله است جلوگیری می‌کند.

- ◊ حالتها: چیدمان n وزیر ($1 \leq n \leq 8$) یکی در هر ستون در n ستون سمت چپ، در حالی که هیچ وزیری به وزیر دیگر حمله نکند.

- ◊ تابع پسین: یک وزیر به هر خانه‌ای در آخرین ستون خالی سمت چپ اضافه کن به نحوی که توسط هیچ وزیر دیگری مورد حمله قرار نگیرد.

این تدوین فضای حالت ۸ وزیر را از 3×10^{14} به فقط 20.57 کاهش می‌دهد و یافتن راه‌حلها ساده می‌شود. برای ۱۰۰ وزیر تدوین به روش اولی حدوداً 10^{100} حالت دارد در حالی که تدوین به روش دوم در حدود 10^{52} حالت خواهد داشت (تمرین ۳-۵). کاهش فوق، بسیار چشم‌گیر می‌باشد، ولی فضای حالت دومی باز هم برای الگوریتمهای این فصل بیش از اندازه بزرگ می‌باشد. فصل ۴ تدوین فضای کامل را توضیح می‌دهد و فصل ۵ الگوریتم ساده‌ای را ارائه می‌کند که حتی حل مسأله یک میلیون وزیر را نیز ساده می‌کند.

مسائل دنیای واقعی

می‌دانیم که مسأله مسیریابی چگونه بر اساس محلهای مشخص و جابه‌جایی بین آنها تعریف می‌شود. الگوریتمهای مسیریابی کاربردهای مختلفی از قبیل مسیریابی در شبکه‌های رایانه‌ای، برنامه‌ریزی عملیات نظامی و سامانه‌های برنامه‌ریزی سفرهای خطوط هوایی دارند. تعریف این مسائل معمولاً پیچیده است. مثال ساده‌شده یک مسأله سفر هوایی را به شکل زیر در نظر بگیرید:

- ◊ حالتها: هر کدام با یک محل (مثلاً یک فرودگاه) و زمان جاری مشخص می‌شود.
- ◊ حالت ابتدایی: توسط مسأله مشخص می‌شود.
- ◊ تابع پسین: حالتی را برمی‌گرداند که از انجام یک پرواز زمان‌بندی‌شده (احتمالاً دارای توضیحات بیشتری مانند کلاس صندلی و محل صندلی)، حرکت دیرتر از زمان فعلی به علاوه زمان جابه‌جایی بین فرودگاهها، از فرودگاه فعلی به فرودگاه دیگر، نتیجه می‌شود.

ف $\Rightarrow$ E

Complete-state formulation

تدوین حالت

کامل

Board

تخته

Arrangement

چیدمان

Sequence

توالی

State space

فضای حالت

To reduce

کاهش دادن

Real-world problems

مسائل دنیای واقعی

Route-finding

مسیریابی

Computer networks

شبکه‌های رایانه‌ای

Military operations planning

برنامه‌ریزی عملیات نظامی

Airline travel planning

برنامه‌ریزی سفرهای خطوط هوایی

Scheduled flight

پرواز زمان‌بندی‌شده

Transit time

زمان جابه‌جایی

ف E
Monetary cost
هزینه مالی
Waiting time
زمان انتظار
Flight time
زمان پرواز
Customs
گمرک
Immigration procedures
روالهای مهاجرت
Time of day
زمان روز
Commercial
تجاری
Contingency plan
طرح اقتضایی
Backup reservation
رزرو پشتیبان
Touring problems
مسائل تورینگ
Traveling salesperson problem
مسأله فروشنده
دوره گرد
Improvement
بهبود
Capability
توانایی
Automatic circuit-board drill
سوراخ‌کن
خودکار صفحات
مدار
Layout
چیدمان

◊ آزمون هدف: آیا سر زمان تعیین شده به مقصد رسیده‌ایم؟

◊ هزینه مسیر: به هزینه مالی، زمان انتظار، زمان پرواز، روالهای گمرک و مهاجرت، کیفیت جا، زمان روز، نوع هواپیما و غیره بستگی دارد.

سامانه‌های تجاری مشاوره سفر از همین نوع تدوین مسأله استفاده می‌کنند که البته با پیچیدگیهای اضافی متعددی همراه است که از ساختارهای کرایه عهد دقینوسی که خطوط هوایی تحمیل می‌کنند ناشی می‌شود. کسانی که در تمام فصول از خطوط هوایی استفاده کرده‌اند می‌دانند که تمام سفرها بر اساس برنامه انجام می‌گیرد. یک سامانه واقعاً خوب باید طرحهای اقتضایی برای پیش‌آمدها و احتمالات داشته باشد (از قبیل رزرو پشتیبان در پروازهای دیگر). البته، تا حدی که با توجه به هزینه و احتمال ناکامی برنامه اصلی قابل توجیه باشد.

مسائل تورینگ ارتباط نزدیکی با مسائل مسیریابی دارند، البته با یک تفاوت اساسی. به عنوان مثال، این مسأله را در نظر بگیرید: "از تمام شهرهای شکل ۲-۳ بگذر، درحالی که شروع و پایان سفر در Bucharest است." از دیدگاه مسیریابی، اقدامات با سفرهایی بین شهرهای مجاور متناظر هستند. در عین حال، فضای حالت کاملاً متفاوت است. هر حالت علاوه بر حفظ محل فعلی، باید مجموعه شهرهایی که کارگزار از آن گذشته است را نیز در برداشته باشد. بنابراین، حالت ابتدایی "*In Bucharest; visited {Bucharest}*" خواهد بود. یک حالت میانی می‌تواند به این شکل باشد: "*In Vaslui; visited {Bucharest, Urziceni, Vaslui}*" و آزمون هدف بررسی می‌کند که آیا کارگزار در Bucharest است و از تمامی ۲۰ شهر گذشته است یا نه.

مسأله فروشنده دوره‌گرد (TSP) یک مسأله تورینگ می‌باشد که در آن از هر شهر دقیقاً یک بار باید عبور کنیم. هدف، یافتن کوتاه‌ترین گردش می‌باشد. این مسأله به عنوان یک مسأله NP-سخت شناخته می‌شود. با این حال، تلاشهای بسیار زیادی برای بهبود تواناییهای الگوریتمهای TSP انجام گردیده است. این الگوریتمها علاوه بر برنامه‌ریزی سفرهای فروشنده دوره‌گرد، در کارهایی مانند برنامه‌ریزی جابه‌جاییهای یک دستگاه سوراخ‌کن خودکار صفحات مدار و سامانه‌های انبارداری در کارخانه‌ها به کار می‌روند.

در مسأله چیدمان VLSI باید میلیونها قطعه و اتصال با هدف کمینه‌سازی مساحت، تأخیرهای مدار، ظرفیت جذب یا تولید پارازیت و بیشینه‌سازی محصول تولیدی، روی یک تراشه جایابی شود. مسأله چیدمان پس از مرحله طراحی منطقی مطرح و معمولاً به دو بخش تقسیم می‌شود: چیدمان سلول و مسیریابی کانال. در چیدمان سلول، قطعات اولیه مدار در چندین سلول گروه‌بندی می‌شوند و هر سلول وظیفه مشخصی دارد. هر سلول بر روی صفحه، محل ثابتی دارد (اندازه و شکل) و به تعداد مشخصی اتصال به سلولهای دیگر نیاز دارد. هدف، قرار دادن سلولها بر روی تراشه به نحوی است که همپوشانی نداشته باشند و فضای کافی برای قرار گرفتن سیمهای اتصال بین آنها وجود داشته باشد. مسیریابی کانال، برای هر سیم یک مسیر مشخص در فضای خالی بین سلولها پیدا می‌کند. این مسائل جستجو بسیار پیچیده هستند ولی مطمئناً ارزش حل شدن را دارند. در فصل ۴ با الگوریتمهایی آشنا می‌شویم که توانایی حل آنها را دارند.

جهت‌یابی روبات، تعمیم مسأله مسیریابی است که پیش از این توضیح دادیم. یک روبات به جای یک مجموعه گسسته از مسیرها، می‌تواند در یک فضای پیوسته حرکت کند و یک

مجموعه نامتناهی از اقدامات و حالتها داشته باشد. برای یک روبات دایره‌ای که روی یک سطح صاف حرکت می‌کند، فضا دوبعدی است. اگر روبات دست و پا یا چرخه‌هایی داشته باشد که باید پایش شوند، فضای جستجو چندبُعدی می‌شود. در این صورت به فنون پیشرفته‌ای برای متناهی ساختن فضای جستجو نیاز می‌باشد. برخی از این روشها در فصل ۲۵ بررسی خواهند شد. علاوه بر پیچیدگی مسأله، روباتهای واقعی باید از عهده خطاهایی که در خواندن حس‌گرها و پایشهای موتور به وجود می‌آید نیز برآیند.

تعیین خودکار ترتیب مونتاژ اشیای پیچیده توسط یک روبات، اولین بار توسط *FREDDY (Michie, 172)* نمایش داده شد. از آن زمان به بعد پیشرفت به کندی انجام شده است، ولی مطمئناً به حدی رسیده است که مونتاژ اشیای پیچیده و نودرتویی مانند مونترهای الکتریکی از نظر اقتصادی امکان‌پذیر شده است. در مسائل مونتاژ، هدف پیدا کردن ترتیب مونتاژ قطعات یک شیء می‌باشد. اگر ترتیب اشتباهی انتخاب شود، هیچ راهی برای افزودن بعضی قطعات وجود ندارد مگر این که بخشی از کارهایی که تاکنون انجام شده است را دوباره به حالت اول برگردانیم. بررسی عملی بودن یک گام در ترتیب فوق، یک مسأله جستجوی هندسی پیچیده است که ارتباط نزدیکی با جهت‌یابی روبات دارد. در نتیجه، تولید پسینه‌های مجاز، بخش پرهزینه تعیین ترتیب مونتاژ می‌باشد. در یک الگوریتم عملی، باید از جستجوی کل فضای حالت اجتناب نمود و تنها بخش کوچکی از آن را جستجو کرد. یک مسأله مونتاژ مهم دیگر، طراحی پروتئین است که هدف آن پیدا کردن ترتیب کنار هم قرار گرفتن اسیدهای آمینه‌ای است که یک پروتئین سه‌بُعدی را می‌سازند که ویژگیهای لازم برای مداوای یک بیماری خاص را دارا می‌باشد.

در چند سال اخیر، تقاضای فزاینده‌ای برای روباتهای نرم‌افزاری به وجود آمده است. وظیفه این روباتها جستجو در اینترنت، از قبیل جستجوی پاسخ پرسشها، جستجوی اطلاعات مرتبط و جستجو به منظور خرید می‌باشد. این روباتها کاربرد بسیار خوبی برای فون جستجو می‌باشند. زیرا تصور اینترنت به عنوان یک گراف که از گرههایی (صفحاتی) تشکیل شده که توسط پیوندها به یکدیگر مرتبط هستند، آسان می‌باشد. در فصل ۱۰، جستجو در اینترنت را به طور کامل توضیح می‌دهیم.

۳-۳ جستجوی راه‌حل

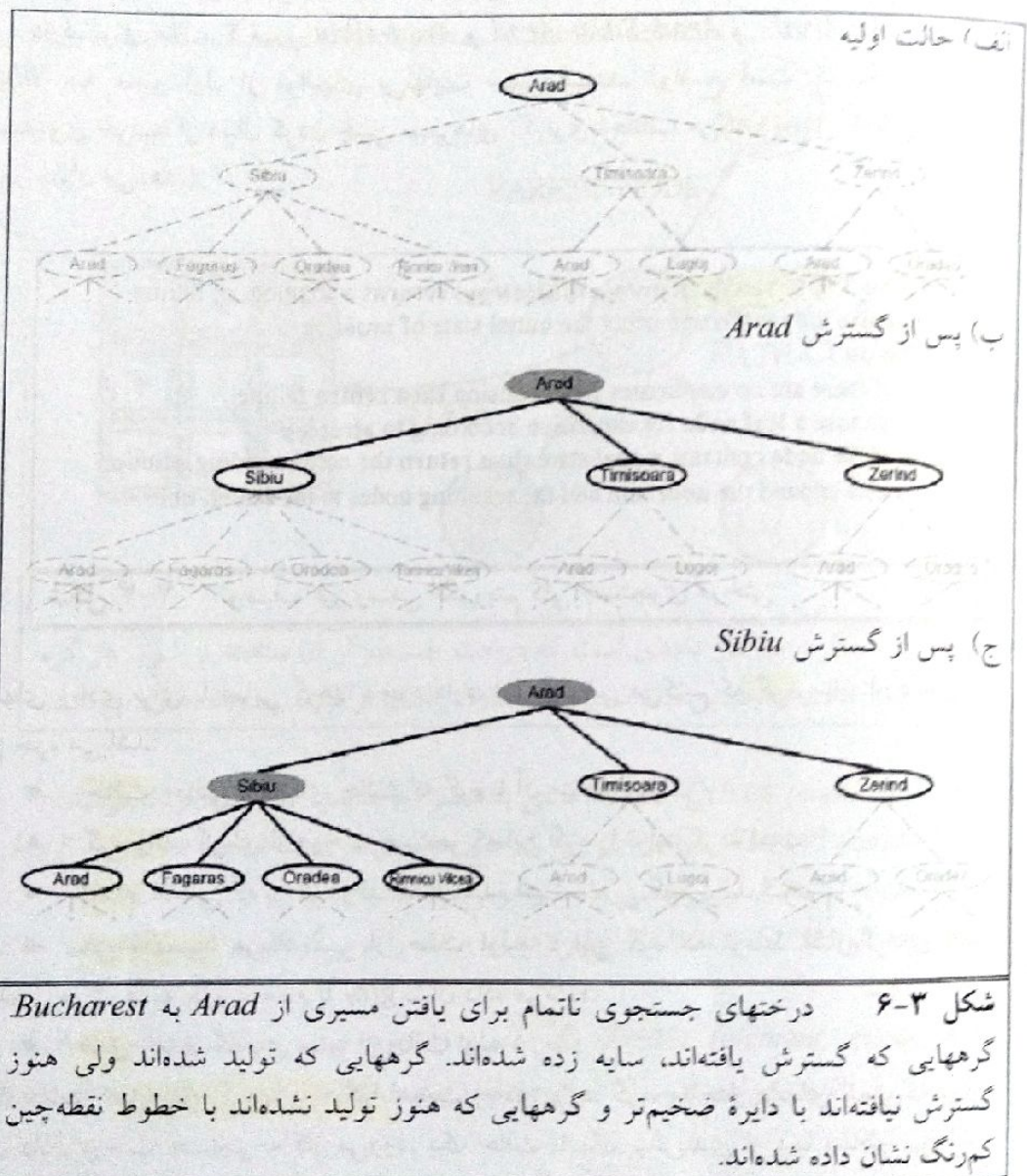
تا اینجا چند مسأله را تدوین کردیم. اکنون باید آنها را حل کنیم. این کار با انجام جستجو در فضای حالت انجام می‌شود. این فصل با فون جستجویی سروکار دارد که از یک درخت جستجوی صریح استفاده می‌کند. این درخت با استفاده از حالت ابتدایی و تابع پسین که همراه یکدیگر فضای حالت را تعریف می‌کنند، ایجاد می‌شود. به طور کلی ممکن است به جای درخت جستجو، گراف جستجو داشته باشیم و آن زمانی است که یک حالت از چند مسیر قابل دسترسی باشد. این مفهوم مهم و پیچیده در بخش ۳-۵ بررسی می‌شود.

شکل ۳-۶ چند نمونه گسترش در درخت جستجو برای یافتن راهی از *Arad* به *Bucharest* را نشان می‌دهد. ریشه درخت جستجو، گره جستجویی است که با حالت ابتدایی

E ⇔ F	سطح صاف
Flat surface	سطح صاف
Two-dimensional	دو بُعدی
Many-dimensional	چند بُعدی
Automatic assembly sequencing	تعیین خودکار ترتیب مونتاژ
Progress	پیشرفت
Electric motors	موتورهای الکتریکی
Geometrical	هندسی
Navigation	جهت‌یابی
Legal successors	پسینه‌های مجاز
Protein design	طراحی پروتئین
Amino acid	اسید آمینه
To cure	مداوا کردن
Disease	بیماری
Formulation	تدوین
Expensive	پرهزینه
Link	پیوند

$E \Rightarrow$	ف
Expansion	گسترش
Arad	آراد
Bucharest	بخارست
Root	ریشه
Tree	درخت
Search node	گره جستجو
To test	آزمودن
Trick problem	مسأله نکته‌دار
To consider	بررسی کردن
Successor function	تابع پسین
Set	مجموعه
To generate	تولید کردن
Partial search tree	درخت جستجوی ناتمام
Dashed line	نقطه چین
Case	مورد
To apply	اعمال کردن
Possibility	امکان
Bold	ضخیم

$In(Arad)$ متناظر است. گام اول، این است که آیا این گره یک حالت هدف است یا نه. واضح است که چنین نیست. ولی این بررسی اهمیت زیادی دارد، زیرا شاید با یک مسأله نکته‌دار مانند "با شروع از $Arad$ به $Arad$ برو" مواجه باشیم. از آنجا که این موقعیت هدف نیست، باید حالت‌های دیگری را مورد بررسی قرار دهیم. این کار با گسترش حالت فعلی انجام می‌شود؛ به این معنی که تابع پسین را به حالت فعلی اعمال می‌کنیم و در نتیجه مجموعه جدیدی از حالت‌ها تولید می‌شود. در این مثال، سه حالت جدید به دست می‌آید: $In(Sibiu)$ ، $In(Timisoara)$ و $In(Zerind)$. اکنون باید تصمیم بگیریم که کدام یک از این سه امکان را در ادامه دنبال نماییم.



اساس جستجو این است که فعلاً یک گزینه را دنبال می‌کنیم و بقیه را برای بعد کنار می‌گذاریم تا در صورتی که گزینه اول به راه‌حل منتج نشد به آنها مراجعه کنیم. فرض کنید که ابتدا $Sibiu$ را انتخاب می‌کنیم. آزمایش می‌کنیم که آیا حالت هدف است (که نیست) و سپس آن را گسترش می‌دهیم و $In(Arad)$ ، $In(Fagaras)$ ، $In(Oradea)$ و $In(RimnicuVilsea)$ را به دست می‌آوریم. سپس می‌توان یکی از این چهار عمل را انتخاب کرد یا با برگشت به عقب،

Timisoara یا Zerind را انتخاب کنیم. با انتخاب، آزمایش و گسترش ادامه می‌دهیم تا آن ی یک راه حل پیدا شود یا هیچ حالت دیگری برای گسترش وجود نداشته باشد. این که کدام حالت را برای گسترش انتخاب کنیم، توسط **راهبرد جستجو** تعیین می‌شود. الگوریتم جستجوی درختی به طور غیر رسمی در شکل ۳-۷ توصیف شده است.

تفاوت قائل شدن بین فضای حالت و درخت جستجو، اهمیت بسیار زیادی دارد. در فضای حالت مسأله مسیریابی، ۲۰ حالت وجود دارد که به هر شهر یکی اختصاص می‌یابد. اما تعداد بی‌شماری مسیر در این فضای حالت وجود دارد، در نتیجه درخت جستجو تعداد بی‌نهایتی گره دارد. برای مثال، ۳ مسیر *Arad-Sibiu-Arad* و *Arad-Sibiu-Arad* و *Arad-Sibiu-Arad* سه مسیر اول از توالیهای بی‌نهایت مسیر هستند. (واضح است که یک الگوریتم جستجوی خوب، از دنبال کردن چنین مسیرهای تکراری اجتناب می‌کند؛ بخش ۳-۵ چگونگی آن را نشان می‌دهد.)

function **TREE-SEARCH**(*problem, strategy*) returns a solution, or failure
 initialize the search tree using the initial state of *problem*
 loop do
 if there are no candidates for expansion then return failure
 choose a leaf node for expansion according to *strategy*
 if the node contains a goal state then return the corresponding solution
 else expand the node and add the resulting nodes to the search tree

شکل ۳-۷ توصیف غیر رسمی الگوریتم کلی جستجوی درختی

راههای زیادی برای بازنمایی گرهها وجود دارد، اما ما فرض می‌کنیم که **گره**، ساختار داده‌ای با پنج جزء می‌باشد:

- **حالت:** حالتی از فضای حالت که گره با آن متناظر است.
- **گره والد:** گره‌ای از درخت جستجو که این گره را تولید کرده است.
- **اقدام:** اقدامی که بر گره والد اعمال شده است تا این گره تولید شود.
- **هزینه مسیر:** هزینه مسیر از حالت اولیه تا این گره که توسط اشاره‌گرهای والد مشخص می‌شود. به طور مرسوم با $g(n)$ نشان داده می‌شود.
- **عمق:** تعداد گامهای مسیر از حالت اولیه

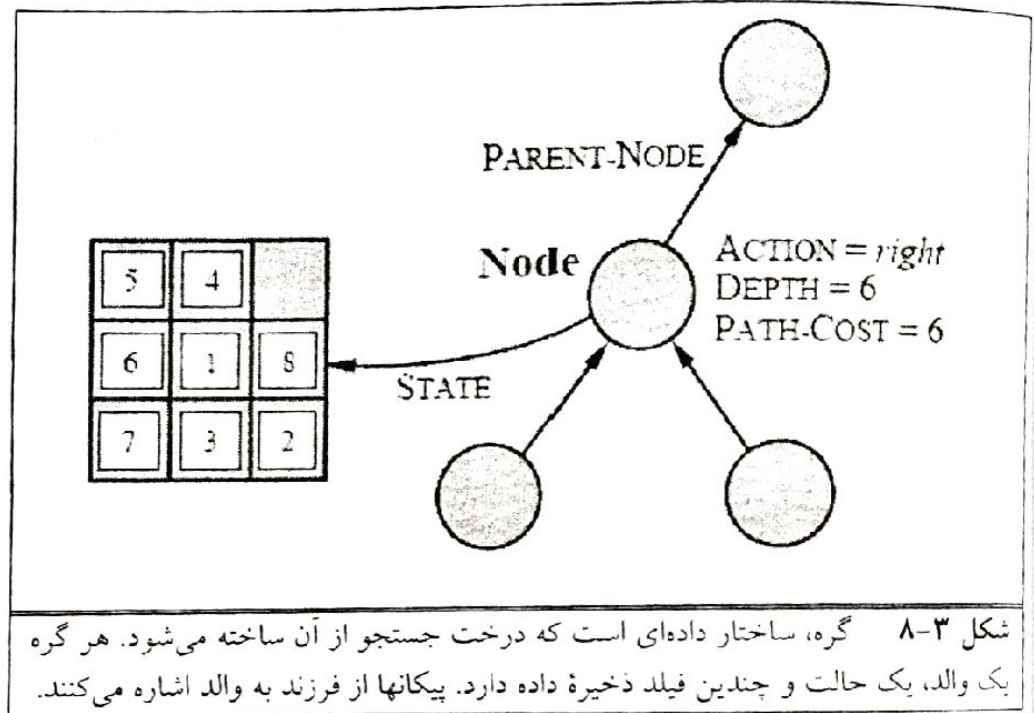
به یاد داشتن تمایز بین گرهها و حالتها اهمیت زیادی دارد. گره، ساختار داده‌ای است که برای نشان دادن درخت جستجو به کار می‌رود. یک حالت با یک پیکربندی از دنیا متناظر می‌باشد. بنابراین، گرهها بر اساس اشاره‌گرهای گره والد، بر روی مسیرهای خاصی قرار دارند، در حالی که حالتها این گونه نیستند. علاوه بر این، دو گره متفاوت می‌توانند حالت دنیای یکسانی داشته باشند، اگر آن حالت از طریق دو مسیر جستجوی مختلف تولید شود. ساختار داده گره در شکل ۳-۸ به تصویر کشیده شده است.

همچنین باید مجموعه گرههایی را که تولید شده‌اند اما هنوز گسترش نیافته‌اند را نیز نشان دهیم (این مجموعه را **لبه می‌نامیم**). هر عنصر **لبه**، گره برگی نامیده می‌شود و گرهی است که

E	ف
Essence	اساس
Option	گزینه
The others	بقیه
To suppose	فرض کردن
To choose	انتخاب کردن
To test	آزمایش کردن
To expand	گسترش دادن
Search strategy	راهبرد جستجو
Tree-search algorithm	الگوریتم جستجوی درختی
Informal	غیر رسمی
To describe	توصیف کردن
City	شهر
Infinite	بی‌نهایتی
Data structure	ساختار داده
Parent	والد
Pointer	اشاره‌گر
Depth	عمق
Configuration	پیکربندی

ف $\rightarrow$ E
 Fringe
 لبه
 Leaf node
 گره برگ
 Search strategy
 راهبرد جستجو
 Circle
 دایره
 To represent
 بازنمایی کردن
 Set
 مجموعه
 To insert
 درج کردن
 Conceptually
 از نظر مفهومی
 Expensive
 پرهزینه
 Queue
 صف
 Arrow
 پیکان
 Bookkeeping field
 فیلد ذخیره داده
 Element
 عنصر
 To return
 برگرداندن
 To remove
 حذف کردن
 Resulting queue
 صف حاصل
 Order
 ترتیب
 First
 اولین
 To implement
 پیاده‌سازی کردن

هیچ پسینی در درخت ندارد. در شکل ۳-۶ لبه هر درخت، از گره‌هایی تشکیل می‌شود که دایره ضخیمی دارند. ساده‌ترین روش بازنمایی لبه، استفاده از یک مجموعه گره خواهد بود. راهبرد جستجو نیز تابعی خواهد بود که گره بعدی که باید گسترش یابد را از این مجموعه انتخاب می‌کند. اگرچه این موضوع از نظر مفهومی ساده و سراسر است، با این حال ممکن است از نظر محاسباتی پرهزینه باشد، زیرا ممکن است تابع راهبرد برای انتخاب بهترین، مجبور به بررسی تک تک عناصر مجموعه شود. بنابراین، فرض می‌کنیم که مجموعه گره‌ها به شکل یک صف پیاده‌سازی می‌شوند. اعمالی که بر روی صف انجام می‌شوند به شرح زیر است:



- *MAKE-QUEUE (element...)*: صفی با عنصر (یا عناصر) داده شده تولید می‌کند.
- *EMPTY? (queue)*: تنها در صورتی که هیچ عنصری در صف نباشد، *true* برمی‌گرداند.
- *FIRST (queue)*: اولین عنصر صف را برمی‌گرداند.
- *REMOVE-FIRST (queue)*: اولین عنصر صف را برمی‌گرداند و آن را از صف حذف می‌کند.
- *INSERT (element, queue)*: یک عنصر را وارد صف می‌کند و صف حاصل را برمی‌گرداند. (خواهیم دید که انواع مختلف صف، عناصر را به ترتیبهای مختلف اضافه می‌کنند).
- *INSERT-ALL (elements, queue)*: یک مجموعه از عناصر را وارد صف می‌کند و صف حاصل را برمی‌گرداند.

با توجه به این تعاریف، می‌توان نسخه رسمی‌تری از الگوریتم عمومی جستجوی درختی نوشت که در شکل ۳-۹ نیز نمایش داده شده است.

اندازه‌گیری کارایی حل مسئله

خروجی یک الگوریتم حل مسئله، یا شکست است یا یک راه‌حل. (بعضی از الگوریتمها ممکن است در یک حلقه بی‌نهایت گیر کنند و هرگز خروجی برنگردانند.) کارایی یک الگوریتم را از چهار زاویه ارزیابی می‌کنیم:

- ◊ کامل بودن: آیا هنگامی که جوابی وجود دارد، این الگوریتم پیدا کردن جواب را تضمین می‌کند؟
- ◊ بهینگی: آیا این راهبرد، جواب بهینه را که در صفحه ۶۲ تعریف شده است پیدا می‌کند؟
- ◊ پیچیدگی زمانی: یافتن یک جواب چه اندازه طول می‌کشد؟
- ◊ پیچیدگی فضایی: برای انجام جستجو، به چه اندازه حافظه نیاز داریم؟

E ⇔ F

General

عمومی

Measuring

اندازه‌گیری

Failure

شکست

Solution

راه‌حل

Infinite loop

حلقه بی‌نهایت

To get stuck

گیر کردن

Complete

کامل

Optimality

بهینگی

To guarantee

تضمین کردن

Strategy

راهبرد

Time complexity

پیچیدگی زمانی

Space complexity

پیچیدگی فضایی

Memory

حافظه

Argument

آرگومان

To note

توجه داشتن

Empty queue

صف خالی

Sequence of actions

توالی اقدامات

Performance

کارایی

function TREE-SEARCH(*problem*, *fringe*) returns a solution, or failure

fringe ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*);

loop do

if EMPTY?(*fringe*) then return failure

node ← REMOVE-FIRST(*fringe*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

fringe ← INSERT-ALL(EXPAND(*node*, *problem*), *fringe*)

function EXPAND(*node*, *problem*) returns a set of nodes

successors ← the empty set

for each (action, result) in SUCCESSOR-FN[*problem*](STATE[*node*]) do

s ← a new NODE

STATE[*s*] ← result

PARENT-NODE[*s*] ← *node*

ACTION[*s*] ← action

PATH-COST[*s*] ← PATH-COST[*node*] + STEP-COST(*node*, action, *s*)

DEPTH[*s*] ← DEPTH[*node*] + 1

add *s* to *successors*

return *successors*

شکل ۳-۹ الگوریتم عمومی جستجوی درختی. (توجه داشته باشید که آرگومان *fringe* باید یک صف خالی باشد و نوع صف، بر ترتیب جستجو اثر می‌گذارد.) تابع SOLUTION توالی اقداماتی را برمی‌گرداند که با دنبال کردن اشاره‌گرهای والد تا ریشه به دست می‌آید.

پیچیدگی زمانی و فضایی همیشه با توجه به درجه سختی مسئله در نظر گرفته می‌شوند. در علم نظری رایانه، مقیاس معمول، اندازه گراف فضای حالت می‌باشد، زیرا گراف به عنوان یک ساختار داده صریح که ورودی برنامه جستجو است در نظر گرفته می‌شود. (نقشه رومانی را به عنوان نمونه در نظر بگیرید.) در هوش مصنوعی، گراف با استفاده از حالت اولیه و تابع پسین

$E \Rightarrow$ ف	درجه سختی
Measure of difficulty	نقشه رومانی
Map of Romania	Infinite
نقشه رومانی	نامتناهی
Branching factor	ضریب انشعاب
ضریب انشعاب	To assess the effectiveness
To assess the effectiveness	برآورد کارایی
برآورد کارایی	To include
To include	شامل شدن
شامل شدن	Official
Official	رسمی
رسمی	To convert
To convert	تبدیل کردن
تبدیل کردن	Estimate
Estimate	تخمین
تخمین	Reasonable
Reasonable	معقول
معقول	Tradeoff point
Tradeoff point	نقطه مصالحه
نقطه مصالحه	Uninformed
Uninformed	ناآگاه
ناآگاه	Blind
Blind	کور
کور	Informed
Informed	آگاه
آگاه	Heuristic
Heuristic	آروین، آروینی
آروین، آروینی	Breadth-first search
Breadth-first search	جستجوی اول
جستجوی اول	سطح
سطح	

به طور صریح نشان داده می‌شود که اغلب نیز نامتناهی است و پیچیدگی نیز با سه کمیت نشان داده می‌شود: b ضریب انشعاب یا حداکثر تعداد پسونهای هر گره: d ، عمق کم عمق‌ترین گره هدف و m طولانی‌ترین مسیر در فضای حالت.

اغلب زمان را بر اساس تعداد گرههایی که در حین جستجو تولید شده‌اند و فضا را با حداکثر گرههای ذخیره شده در حافظه اندازه‌گیری می‌کنند.^۵

برای برآورد کارایی یک الگوریتم جستجو، می‌توان تنها هزینه جستجو (که عموماً به پیچیدگی زمانی بستگی دارد ولی در عین حال می‌تواند میزان استفاده از حافظه را هم شامل شود) را در نظر گرفت یا می‌توان از هزینه کل استفاده کرد که هزینه جستجو و هزینه مسیر راه‌حل یافته‌شده را با هم ترکیب می‌کند. در مورد مسأله پیدا کردن مسیر از Arad به Bucharest، هزینه جستجو مقدار زمانی است که صرف جستجو شده است و هزینه راه‌حل، کل طول مسیر به کیلومتر می‌باشد. بنابراین، برای محاسبه هزینه کل، باید کیلومتر و میلی‌ثانیه را با هم جمع کنیم. هیچ "روش رسمی" برای تبدیل این دو واحد وجود ندارد ولی در این مورد، تبدیل کیلومتر به میلی‌ثانیه با استفاده از تخمین سرعت خودرو می‌تواند معقول باشد (زیرا زمان، چیزی است که برای کارگزار اهمیت دارد). به این ترتیب، کارگزار می‌تواند یک نقطه مصالحه بهینه پیدا کند که در آن نقطه، محاسبات بیشتر برای یافتن مسیر کوتاهتر، اثر معکوس دارد. مسأله عمومی‌تر مصالحه بین چیزهای مختلف در فصل ۱۶ بحث می‌شود.

۴-۳ راهبردهای جستجوی ناآگاهانه

این بخش شامل پنج راهبرد جستجو است که با عنوان جستجوی ناآگاهانه (یا جستجوی کور) مطرح می‌شوند. این اصطلاح بدین معنی است که آنها هیچ اطلاعات اضافی به جز آنچه در تعریف مسأله آمده است درباره حالت ندارند. این راهبردها تنها قادر به تولید پسونها و تشخیص یک حالت هدف از یک حالت غیرهدف می‌باشند. راهبردهایی که می‌دانند یک حالت غیرهدف، بحث موفقیت بیشتری نسبت به دیگران دارد راهبردهای جستجوی آگاهانه یا جستجوی آروینی نامیده می‌شوند. در فصل ۴ این راهبردها را بررسی می‌کنیم. تمامی راهبردهای جستجو، بر اساس ترتیب گسترش گرهها، از یکدیگر متمایز می‌شوند.

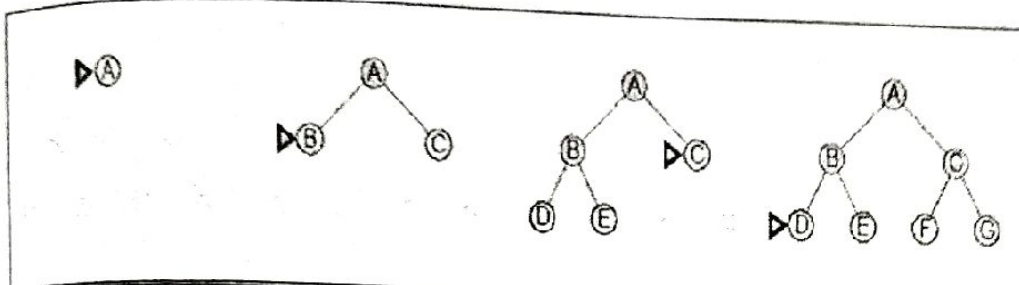
جستجوی اول سطح

جستجوی اول سطح، راهبرد ساده‌ای است که در آن ابتدا ریشه گسترش می‌یابد، سپس تمامی پسونهای گره ریشه گسترش می‌یابند، آنگاه پسونهای آنها گسترش می‌یابند و به طور کلی باید تمامی گرههای یک سطح از درخت جستجو گسترش یابند تا گرهی در سطح بعدی بتواند گسترش یابد.

جستجوی اول سطح را می‌توان با فراخوانی TREE-SEARCH با یک لبه خالی که یک

^۵ برخی از کتابها زمان را بر اساس تعداد گسترشهای گره اندازه‌گیری می‌کنند. این دو مقیاس حداکثر به اندازه ضریب b با یکدیگر تفاوت دارند. به نظر می‌رسد که زمان اجرای گسترش یک گره، با تعداد گرههایی که در آن گسترش تولید می‌شوند، افزایش می‌یابد.

صف **FIFO** (اولین ورودی، اولین خروجی) است پیاده‌سازی کرد. در **FIFO** گره‌هایی که اول از همه دیده شده‌اند، اول گسترش خواهند یافت. به عبارت دیگر، فراخوانی **TREE-SEARCH** تمامی صف **FIFO** تمامی پسین‌هایی که تازه تولید می‌شوند را در انتهای صف قرار می‌دهد، بدین معنی که گره‌های کم‌عمق زودتر از گره‌های عمیق‌تر گسترش می‌یابند. شکل ۳-۱۰ نحوه پیشرفت جستجو را در یک درخت دودویی ساده نشان می‌دهد.



شکل ۳-۱۰ جستجوی اول‌سطح در یک درخت دودویی ساده. در هر مرحله گرهی که باید در مرحله بعدی گسترش یابد با یک مثلث مشخص شده است.

اکنون جستجوی اول‌سطح را به کمک چهار معیاری که در قسمت قبل گفتیم ارزیابی می‌کنیم. می‌توان به آسانی مشاهده کرد که این جستجو کامل است، به این دلیل که اگر کم‌عمق‌ترین گره هدف در عمق متناهی d باشد، جستجوی اول‌سطح نهایتاً بعد از گسترش تمامی گره‌های کم‌عمق‌تر، آن را خواهد یافت (به شرطی که ضریب انشعاب b متناهی باشد). کم‌عمق‌ترین گره هدف، ضرورتاً بهینه‌ترین نیست. از نظر فنی، جستجوی اول‌سطح در صورتی بهینه است که هزینه مسیر یک تابع غیر کاهشی از عمق گره باشد (برای مثال، وقتی که تمامی اقدامات هزینه یکسانی دارند).

تا اینجا، مطالب امیدوارکننده‌ای درباره جستجوی اول‌سطح گفته شد. برای این که متوجه شویم چرا جستجوی اول‌سطح همیشه به عنوان راهبرد مطلوب انتخاب نمی‌شود، باید به مقدار زمان و حافظه‌ای که برای انجام یک جستجو صرف می‌شود، توجه کرد. برای انجام این کار فضای حالت فرضی را در نظر می‌گیریم که در آن هر حالت، b پسین دارد. ریشه درخت جستجو در سطح اول b گره تولید می‌کند که هر کدام از آنها b گره دیگر تولید می‌کنند. بنابراین در سطح دوم کلاً b^2 گره داریم. هر کدام از اینها، b گره دیگر تولید می‌کنند که در سطح سوم b^3 گره می‌شود و الی آخر. اکنون، فرض کنید که راه‌حل در عمق d قرار دارد. در بدترین حالت، تمامی گره‌های سطح d به جز یکی را (از آنجا که خود هدف گسترش نمی‌یابد) گسترش داده‌ایم و در سطح $d+1$ ، $b^{d+1}-b$ گره تولید کرده‌ایم. در آن صورت تعداد کل گره‌هایی که تولید شده‌اند به شرح زیر می‌باشد.

$$b + b^2 + b^3 + \dots + b^d + (b^{d+1} - b) = O(b^{d+1})$$

هر گرهی که تولید شده است باید در حافظه باقی بماند، زیرا یا جزئی از لبه یا از اجداد یک گره لبه است. در نتیجه، پیچیدگی فضایی مانند پیچیدگی زمانی است (به علاوه یک گره برای

ف $\Rightarrow$ E
To be expanded
گسترش یافتن
First-in-first-out (FIFO)
اولین ورودی، اولین خروجی
To call
فراخوانی کردن
Binary tree
درخت دودویی
Shallow
کم‌عمق
Triangle
مثلث
Complete
کامل
Branching factor
ضریب انشعاب
Non-decreasing function
تابع غیر کاهشی
Strategy
راهبرد
Desired
مطلوب
Successor
پسین
Root
ریشه
Level
سطح
Depth
عمق
Worst case
بدترین حالت

$E \Rightarrow$	ف
Fringe	لبه
Ancestors	اجداد
Node	گره
Space complexity	پیچیدگی فضایی
Complexity analysis	تحلیل پیچیدگی
Branching factor	ضریب انشعاب
Factor	ضریب
Fortunately	خوشبختانه
Exponential complexity	پیچیدگی نمایی
Uninformed methods	روشهای ناآگاهانه
Uniform-cost search	جستجوی هزینه یکنواخت
Optimal	بهینه
Extension	بسط
Step cost function	تابع هزینه مرحله

ریشه).

کسانی که تحلیل پیچیدگی انجام می دهند، حدود پیچیدگی نمایی، مانند $O(b^{d+1})$ آنها را نگران (با اگر به چالش علاقمند باشند، هیجان زده) می کند. شکل ۳-۱۱ علت آن را توضیح می دهد. این شکل، زمان و حافظه مورد نیاز جستجوی اول سطح را با ضریب انشعاب $b=10$ و مقادیر مختلف عمق راه حل d نشان می دهد. در این جدول فرض می شود که در هر ثانیه ۱۰۰۰۰ گره می تواند تولید شود و هر گره ۱۰۰۰ بایت برای ذخیره سازی نیاز دارد. این فرضیات تقریباً در مورد اکثر مسائل جستجو که در یک رایانه امروزی اجرا شوند صادق است (با افزودن یا حذف یک ضریب ۱۰۰).

از شکل ۳-۱۱ دو نکته می توان آموخت. اول آن که در جستجوی اول سطح نیاز به حافظه، مشکل بزرگ تری نسبت به زمان اجراست. ۳۱ ساعت انتظار برای حل یک مسأله مهم در عمق ۸ خیلی زیاد نیست، ولی تنها رایانه های معدودی وجود دارند که حافظه اصلی یک ترابایتی مورد نیاز را داشته باشند. خوشبختانه، راهبردهای جستجوی دیگری وجود دارد که به حافظه کمتری احتیاج دارند.

نکته دوم این است که نیازهای زمانی هم عامل مهمی به شمار می روند. اگر راه حل مسأله شما در عمق ۱۲ باشد، آنگاه (بر اساس فرضیات ما)، جستجوی اول سطح (یا در حقیقت هر جستجوی ناآگاهانه ای) به ۳۵ سال زمان برای یافتن راه حل نیاز دارد. به طور کلی، مسائل جستجوی با پیچیدگی نمایی، به جز برای نمونه های کوچک، توسط روشهای ناآگاهانه قابل حل نیستند.

حافظه	زمان	گره	عمق
1 megabyte	11 seconds	1100	2
106 megabytes	11 seconds	111,100	4
10 gigabytes	19 minutes	10^7	6
1 terabytes	31 hours	10^9	8
101 terabytes	129 days	10^{11}	10
10 petabytes	35 years	10^{13}	12
1 exabyte	3.523 years	10^{15}	14

شکل ۳-۱۱ زمان و حافظه مورد نیاز جستجوی اول سطح. فرض شده است که ضریب انشعاب $b=10$ ، ۱۰۰۰۰ گره در ثانیه، ۱۰۰۰ بایت بر گره می باشد.

جستجوی هزینه یکنواخت

جستجوی اول سطح هنگامی بهینه است که هزینه تمام مراحل یکسان باشد. زیرا همیشه کم عمق ترین گرهی که گسترش نیافته است را گسترش می دهد. با افزودن یک بسط ساده، می توانیم الگوریتمی بیابیم که برای هر تابع هزینه مرحله، بهینه است. جستجوی هزینه

یکنواخت به جای گسترش کم عمق ترین گره، گره ۱۱ را که کمترین هزینه مسیر را دارد گسترش می دهد. توجه کنید که اگر تمامی هزینه های مراحل مختلف مساوی باشد، این جستجو با جستجوی اول سطح یکسان می شود.

جستجوی هزینه یکنواخت، به تعداد مراحل یک مسیر اهمیت نمی دهد، بلکه فقط هزینه کل آنها را در نظر می گیرد. در نتیجه، اگر گرهی را گسترش دهد که دارای اقدامی با هزینه صفر باشد و به همان حالت برگردد (مانند یک اقدام *NoOp*)، این جستجو در یک حلقه بی نهایت گیر خواهد افتاد. در صورتی می توانیم کامل بودن را تضمین کنیم که هزینه هر مرحله، بزرگتر یا مساوی یک مقدار ثابت کوچک مثبت مانند ϵ باشد. این شرط برای تضمین بهینگی نیز کافی است. این بدان معنی است که همیشه با جلو رفتن در مسیر، هزینه مسیر افزایش می یابد. بر اساس این ویژگی به سادگی می توان نتیجه گرفت که این الگوریتم، گره ها را به ترتیب افزایش هزینه مسیر گسترش می دهد. (به خاطر داشته باشید که *TREE-SEARCH* آزمون هدف را تنها در مورد گره هایی اعمال می کند که برای گسترش انتخاب شده اند.) پیشنهاد می شود که این الگوریتم را برای پیدا کردن کوتاه ترین مسیر به *Bucharest* امتحان کنید.

جستجوی هزینه یکنواخت به جای عمق بر اساس هزینه مسیر هدایت می شود. در نتیجه، پیچیدگی آن بر اساس d و b به سادگی قابل بیان نیست. در عوض، اگر C^* هزینه راه حل بهینه باشد و فرض کنیم که هر اقدامی، حداقل ϵ هزینه دارد، آنگاه این الگوریتم در بدترین حالت دارای پیچیدگی زمانی و فضایی $O(b^{C^*/\epsilon})$ خواهد بود که می تواند بسیار بزرگتر از b^d باشد. علت آن این است که جستجوی هزینه یکنواخت اغلب درخت های بزرگی که دارای مراحل کوچکی هستند را قبل از مسیرهای دارای مراحل بزرگ و احتمالاً سودمند، کاوش می کند. واضح است که اگر تمامی هزینه های مراحل یکسان باشد $b^{C^*/\epsilon}$ معادل b^d خواهد بود.

جستجوی اول عمق

جستجوی اول عمق همیشه عمیق ترین گره در لبه فعلی از درخت جستجو را گسترش می دهد. فرایند جستجو در شکل ۳-۱۲ نشان داده شده است. جستجو بلافاصله به عمیق ترین سطح درخت جستجو هدایت می شود که در آنجا گره ها هیچ پسینی ندارند. در همان حال که گره ها گسترش می یابند، از لبه خارج می شوند. بنابراین، جستجو به عمیق ترین گره بعدی برمی گردد که هنوز پسینهای کاوش نشده دارد.

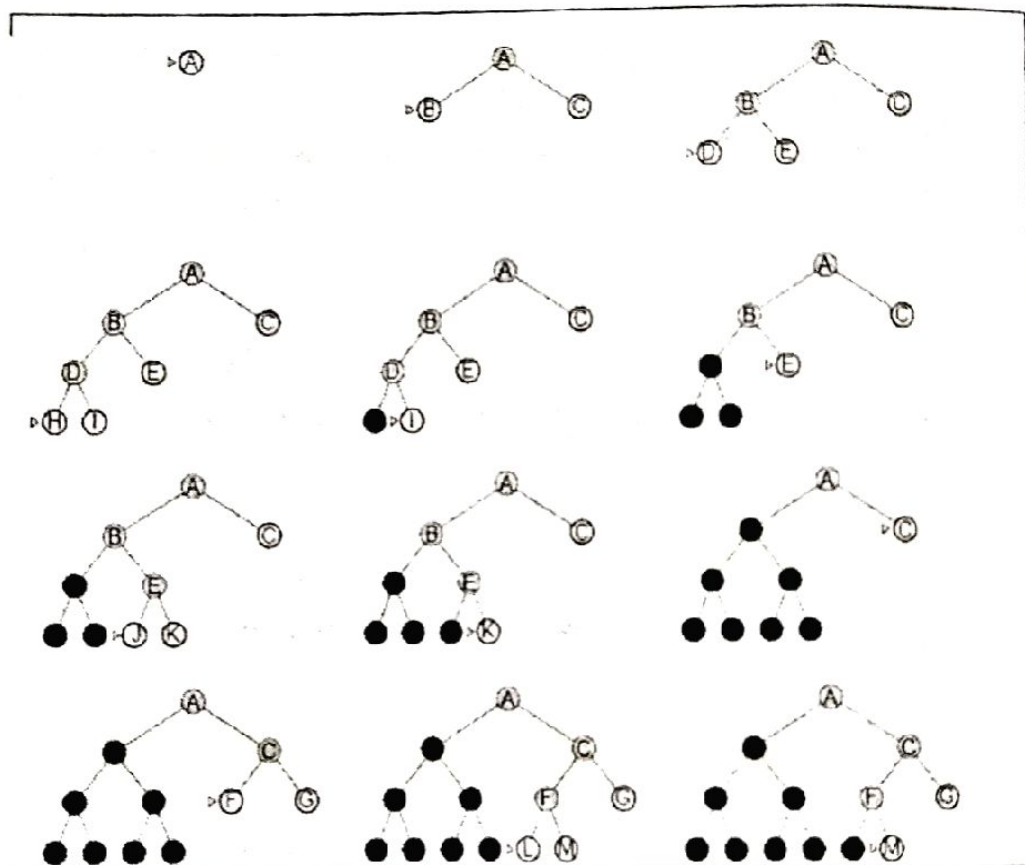
این راهبرد، به کمک *TREE-SEARCH* همراه با یک صف آخرین ورودی، اولین خروجی (*LIFO*) که به آن پشته هم می گوئیم قابل پیاده سازی است. معمولاً به عنوان یک روش جایگزین برای پیاده سازی *TREE-SEARCH*، از یک تابع بازگشتی که در هر یک از فرزندان، به ترتیب خودش را فراخوانی می کند، برای پیاده سازی جستجوی اول عمق استفاده می شود. (یک الگوریتم اول عمق بازگشتی که محدودیت عمق دارد در شکل ۳-۱۳ نمایش داده شده است.)

جستجوی اول عمق، به حافظه نسبتاً کمی نیاز دارد. این الگوریتم تنها نیاز دارد که یک مسیر از ریشه تا گره برگ، همراه با گره های هم نیای گسترش نیافته تمامی گره های آن مسیر را نگهداری کند. اگر گرهی گسترش پیدا کرد، به محض این که تمامی فرزندان کاملاً کاوش شدند، می تواند از حافظه حذف شود. (شکل ۳-۱۲ را نگاه کنید.) در مورد فضای حالتی با

ف E	Shallowest
کم عمق ترین	Equal
مساوی	Identical
یکسان	Infinite loop
حلقه بی نهایت	To get stuck
گیر افتادن	Constant value
مقدار ثابت	Guarantee
تضمین	To remember
به خاطر داشتن	Goal test
آزمون هدف	Shortest path
کوتاه ترین مسیر	Depth-first search
جستجوی اول عمق	Recursive
بازگشتی	Level
سطح	Successor
پسین	Unexplored
کاوش نشده	LIFO
آخرین ورودی	اولین خروجی
اولین خروجی	Stack
پشته	

$E \Rightarrow$ ف	
Leaf node	گره برگ
Sibling nodes	گره‌های هم‌نیا
Goal node	گره هدف
Successor	پسین
Petabyte	پتابایت
Binary tree	درخت دودویی
To remember	به خاطر سپردن
Fringe	لبه
To remove	حذف کردن
Backtracking search	جستجوی پس‌گرد
Partially expanded	نیمه گسترش یافته
Trick	ترفند
Memory-saving	صرفه‌جویی در حافظه
Next	بعدی
Child	فرزند
Path	مسیر

ضریب انشعاب b حداکثر عمق m جستجوی اول عمق به ذخیره‌سازی تنها $bm+1$ گره نیاز دارد. با در نظر گرفتن فرضیاتی مشابه با فرضیات شکل ۳-۱۱ و همچنین این فرض که گره‌هایی که در یک سطح به عنوان گره هدف قرار دارند، هیچ پسینی ندارند، درمی‌یابیم که جستجوی اول عمق، در عمق $d=12$ به جای ۱۰ پتابایت به ۱۱۸ کیلوبایت نیاز دارد (یعنی ۱۰ میلیارد بار فضای کمتر).



شکل ۳-۱۲ جستجوی اول عمق بر روی یک درخت دودویی. می‌توان گره‌هایی که گسترش یافته‌اند و هیچ فرزندی در لبه ندارند را از حافظه حذف کرد. این گره‌ها با رنگ سیاه نشان داده شده‌اند. فرض شده است که گره‌ها در عمق ۳ هیچ پسینی نداشته باشند و M تنها گره هدف باشد.

نوع خاصی از جستجوی اول عمق به نام **جستجوی پس‌گرد** از حافظه کمتری استفاده می‌کند. در جستجوی پس‌گرد، به جای تمام پسین‌ها، در هر لحظه فقط یک پسین تولید می‌شود. هر یک از گره‌های نیمه‌گسترش یافته، به خاطر می‌سپارد که کدام پسین را در مرحله بعدی باید تولید کند. بدین ترتیب، به جای $O(bm)$ به $O(m)$ حافظه نیاز داریم. جستجوی پس‌گرد از ترفند دیگری نیز برای صرفه‌جویی در حافظه (و زمان) بهره می‌برد: تولید پسین با تغییر مستقیم توصیف حالت فعلی، به جای این که ابتدا آن را کپی کنیم. از این طریق، حافظه مورد نیاز تنها به یک توصیف حالت و $O(m)$ اقدام کاهش می‌یابد. برای این که این راهبرد به درستی عمل کند باید این توانایی را داشته باشیم که هر کدام از تغییرات را به حالت اولیه برگردانیم تا بتوانیم

برای تولید پسین بعدی به عقب برگردیم. این فنون برای موفقیت مسائلی مانند مونتاژ روباتیک که توصیف حالتهای آن حجیم است، حیاتی هستند.

اشکال جستجوی اول عمق این است که ممکن است انتخاب غلطی انجام دهد و در پایین رفتن از یک مسیر بسیار طولانی (و حتی نامتناهی) گیر کند، در حالی که یک انتخاب دیگر می توانست به پیدا کردن جوابی در نزدیکی ریشه درخت جستجو منجر شود. برای مثال، در شکل ۱۲-۳ جستجوی اول عمق زیر درخت سمت چپ را به طور کامل کاوش خواهد کرد، حتی اگر گره C یک گره هدف باشد. اگر گره d نیز یک گره هدف بود، آنگاه جستجوی اول عمق از راه به عنوان راه حل برمی گرداند. بنابراین، جستجوی اول عمق بهینه نیست. اگر زیر درخت سمت چپ دارای عمق نامحدود بود و در عین حال شامل هیچ راه حلی نبود، جستجوی اول عمق هرگز پایان نمی یافت. در نتیجه، این روش کامل نیست. در بدترین حالت جستجوی اول عمق تمامی $O(b^m)$ گره را از درخت جستجو تولید می کند. (m حداکثر عمق گره ها است). توجه داشته باشید که m می تواند بسیار بزرگتر از d (عمق کم عمق ترین راه حل) باشد و اگر درخت نامحدود باشد بی نهایت خواهد بود.

جستجوی عمق محدود

مشکل درختان نامحدود را می توان با در نظر گرفتن یک محدودیت عمق از پیش تعیین شده، مانند l تا حدی کاهش داد. این بدان معناست که با گره هایی که در عمق l هستند، به گونه ای رفتار می شود که گویی هیچ پسینی ندارند. به این رویکرد، جستجوی عمق محدود می گویند. محدودیت عمق، مسأله مسیر نامتناهی را حل می کند. متأسفانه اگر $l < d$ انتخاب شود (یعنی کم عمق ترین هدف، پایین تر از محدوده عمق قرار داشته باشد) عامل دیگری برای ناتمام بودن روش ایجاد می شود. هنگامی که مقدار d نامعلوم است، این امکان چندان غیر متحمل نیست. همچنین اگر $l > d$ انتخاب شود، جستجوی عمق محدود غیر بهینه خواهد بود. پیچیدگی زمانی این روش $O(b^l)$ و پیچیدگی فضایی آن $O(bl)$ می باشد. جستجوی اول عمق را می توان به عنوان مورد خاصی از جستجوی عمق محدود دانست که در آن، $l = \infty$ است.

گاهی اوقات، محدودیت عمق بر اساس دانش مسأله تعیین می شود. برای مثال، در نقشه رومانی ۲۰ شهر وجود دارد. بنابراین، می دانیم که اگر راه حلی وجود داشته باشد، در طولانی ترین حالت، دارای عمق ۱۹ است، یعنی $l = 19$ انتخاب ممکن است. در واقع، اگر نقشه را با دقت بیشتری بررسی می کردیم، درمی یافتیم که هر شهر، در حداکثر ۹ مرحله قابل دسترسی از هر شهر دیگر است. این عدد که به عنوان قطر فضای حالت نیز شناخته می شود، محدودیت عمق بهتری فراهم می کند که به جستجوی اول عمق مؤثرتری منجر می شود. با این حال، در مورد اکثر مسائل، تا زمانی که مسأله حل نشود نخواهیم دانست که بهترین محدودیت عمق چه اندازه است.

با اعمال تغییرات ساده ای در الگوریتم کلی جستجوی درخت یا در الگوریتم جستجوی اول عمق بازگشتی می توان جستجوی عمق محدود را پیاده سازی کرد. شبه کد جستجوی عمق محدود بازگشتی در شکل ۱۳-۳ نشان داده شده است. توجه داشته باشید که توقف همراه با عدم موفقیت جستجوی عمق محدود به دو شکل است: برگرداندن مقدار *failure* استاندارد که نشانگر عدم وجود راه حل می باشد؛ و برگرداندن مقدار *cutoff* که نشانگر عدم وجود راه حل در آن محدوده عمق می باشد.

ف E

Description

توصیف

Strategy

راهبرد

Techniques

فنون

Robotic Assembly

مونتاژ روباتیک

To explore

کاوش کردن

Subtree

زیردرخت

Unbounded depth

عمق نامحدود

Complete

کامل

Infinite

بی نهایت

Depth-limited search

جستجوی عمق محدود

To consider

در نظر گرفتن

Limit

محدوده

Unknown

نامعلوم

Incomplete

ناتمام

Special case

مورد خاص

Knowledge

دانش

Diameter

قطر

E → F

Depth-limited search

جستجوی عمیق

محدود

Modification

تغییر

General algorithm

الگوریتم کلی

To Implement

پیاده‌سازی کردن

Pseudocode

شبه‌کد

Recursive

بازگشتی

Failure

عدم موفقیت

Iterative deepening depth-first search

جستجوی

اول عمق

عمیق‌شونده

تکراری

General strategy

راهبرد عمومی

Depth limit

محدودیت عمق

Gradually increasing

در حال افزایش

تدریجی

In combination with

همراه با

Benefit

مزیت

function DEPTH-LIMITED-SEARCH(*problem*, *limit*) returns a solution, or failure cutoff
return RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[*problem*]), *problem*, *limit*)

function RECURSIVE-DLS(*node*, *problem*, *limit*) returns a solution, or failure cutoff

cutoff-occurred? ← false

if GOAL-TEST[*problem*](STATE[*node*]) then return SOLUTION(*node*)

else if DEPTH[*node*] = *limit* then return cutoff

else for each *successor* in EXPAND(*node*, *problem*) do

result ← RECURSIVE-DLS(*successor*, *problem*, *limit*)

 if *result* = cutoff then *cutoff-occurred?* ← true

 else if *result* ≠ failure then return *result*

if *cutoff-occurred?* then return cutoff else return failure

دیدن نودها از چپ
به راست

شکل ۳-۱۳ پیاده‌سازی بازگشتی جستجوی عمیق محدود

جستجوی اول عمق عمیق‌شونده تکراری

جستجوی عمیق‌شونده تکراری (یا جستجوی اول عمق عمیق‌شونده تکراری) یک راهبرد عمومی است که اغلب همراه با جستجوی اول عمق استفاده می‌شود و بهترین محدودیت عمق را می‌یابد. این کار را با افزایش تدریجی محدودیت (اول، بعد ۱، بعد ۲ و ...) تا زمانی که یک هدف پیدا شود، انجام می‌دهد. این موضوع زمانی اتفاق می‌افتد که محدودیت عمق به d ، یعنی عمق کم عمق‌ترین گره هدف، برسد. این الگوریتم در شکل ۳-۱۴ نشان داده شده است.

function ITERATIVE-DEEPENING-SEARCH(*problem*) returns a solution, or failure
inputs: *problem*, a problem

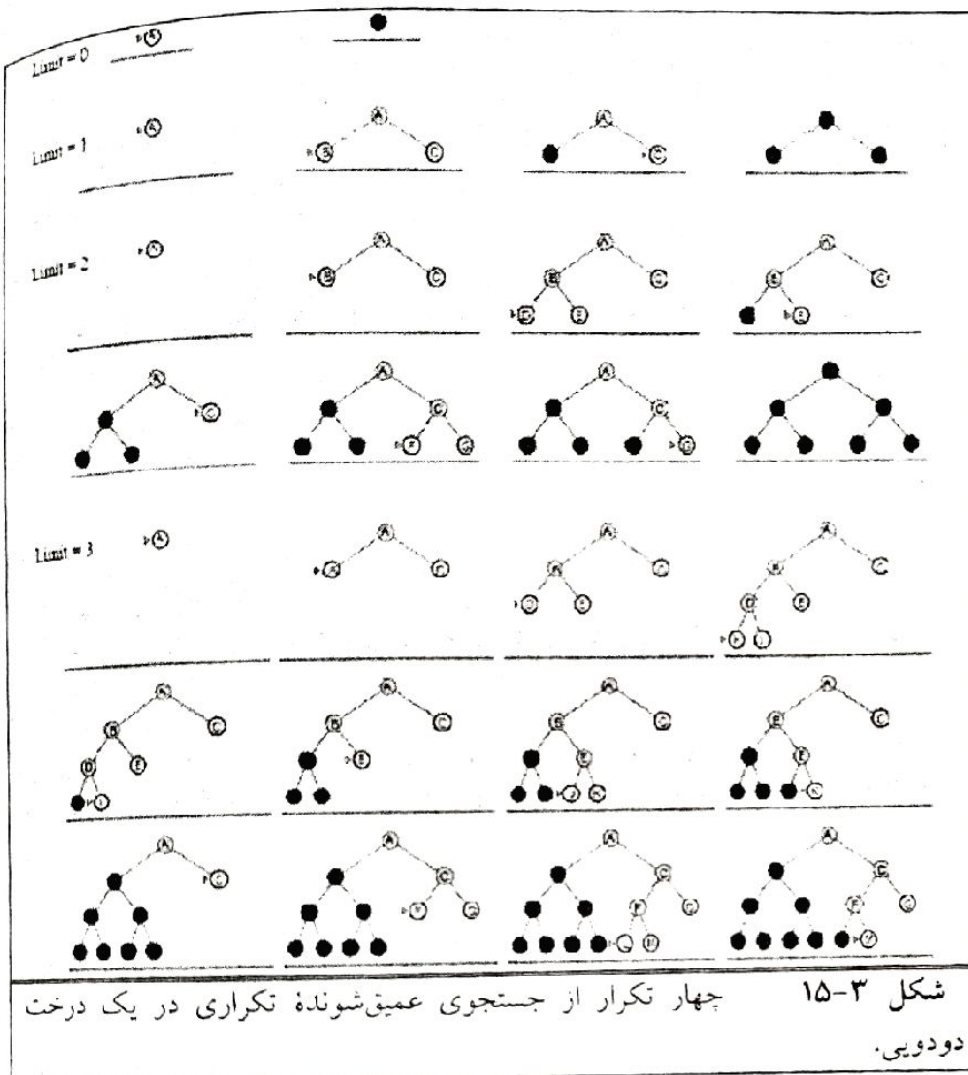
for *depth* ← 0 to ∞ do

result ← DEPTH-LIMITED-SEARCH(*problem*, *depth*)

 if *result* ≠ cutoff then return *result*

شکل ۳-۱۴ الگوریتم جستجوی عمیق‌شونده تکراری که به طور مکرر جستجوی عمق محدود را با محدودیتهای در حال افزایش به کار می‌برد. هنگامی که اتمام می‌رسد که راه‌حلی پیدا نشود و یا جستجوی عمق محدود عدم موفقیت را برگرداند (که به معنی عدم وجود راه‌حل است).

روش عمیق‌شونده تکراری مزایای جستجوی اول عمق و جستجوی اول سطح را یک جا جمع می‌کند. همانند جستجوی اول عمق از حافظه نسبتاً کمی استفاده می‌کند؛ به عبارت دقیقتر، $O(bd)$. همانند جستجوی اول سطح وقتی که ضریب انشعاب محدود است، کامل می‌باشد و نیز وقتی که هزینه مسیر، تابع غیرکاهشی عمق گره باشد، بهینه است. شکل ۳-۱۵ چهار تکرار ITERATIVE-DEEPENING-SEARCH را در یک درخت جستجوی دودویی نشان می‌دهد که در آن راه‌حل در تکرار چهارم پیدا می‌شود.



جستجوی عمیق‌شونده تکراری، ممکن است اسراف‌کارانه به نظر برسد، زیرا حالتها چندین بار تولید می‌شوند. ولی می‌توان نشان داد که این جستجو چندان هم پرهزینه نیست. علت این امر آن است که در یک درخت جستجو که در تمام سطوحش ضریب انشعاب یکسان (یا تقریباً یکسان) است، اکثر گرهها در سطوح پایین‌تر می‌باشند. بنابراین، چندان اهمیت ندارد که سطوح بالاتر چندین بار تولید شوند. در یک جستجوی عمیق‌شونده تکراری، گرههای سطوح پایین‌تر (عمق d) تنها یک بار تولید می‌شوند، گرههایی که در یک سطح بالاتر هستند، دو بار تولید می‌شوند و ... تا فرزندان ریشه که d مرتبه تولید می‌شوند. بنابراین، تعداد کل گرههای تولیدی عبارت است از:

$$N(DFS) = (d)b + (d-1)b^2 + \dots + (1)b^d$$

که دارای پیچیدگی زمانی $O(b^d)$ است. می‌توان این عدد را با گرههای حاصل از جستجوی اول‌سطح مقایسه کرد:

$$N(BFS) = b + b^2 + \dots + b^d + (b^{d+1} - b)$$

 ف $\Rightarrow$ E

Breadth-first search

جستجوی

اول‌سطح

Branching factor

ضریب انشعاب

Non-decreasing

غیرکاهشی

Binary search tree

درخت

جستجوی

دودویی

Iteration

تکرار

Wasteful

اسراف‌کار

Costly

پرهزینه

Level

سطح

The same

یکسان

Time complexity

پیچیدگی زمانی

Depth

عمق

Number

تعداد

To generate

تولید کردن

Node

گره

Root

ریشه

Multiple times

چندین بار

Upper levels

سطوح بالاتر

Despite	علی‌رغم
Repeated	تکراری
Preferred	مقدم، مطلوب
Uninformed search	جستجوی ناآگاهانه
Analogous	قابل قیاس
Layer	لایه
Optimality Guarantee	تضمین بهینگی
To inherit	به ارث بردن
Iterative lengthening search	جستجوی طولانی‌کننده
Overhead	تکراری
Bidirectional search	سربار
Simultaneous	هم‌زمان
Motivation	انگیزه
Circle	دایره
Center	مرکز
Hash table	جدول هش
membership	عضویت

توجه داشته باشید که جستجوی اول‌سطح، چند گره در عمق $d+1$ تولید می‌کند، در حالی که جستجوی عمیق‌شونده تکراری این کار را نمی‌کند. نتیجه این می‌شود که جستجوی عمیق‌شونده تکراری، علی‌رغم تولید تکراری حالتها، سریع‌تر از جستجوی اول‌سطح می‌باشد. برای مثال، اگر $d = 5$ و $b = 10$ باشند خواهیم داشت:

$$N(IDS) = 50 + 400 + 3000 + 20000 + 100000 = 123450$$

$$N(BFS) = 10 + 100 + 1000 + 10000 + 100000 + 999990 = 1111100$$

به طور کلی، زمانی که فضای جستجو بزرگ و عمق راه‌حل نامعلوم است، جستجوی عمیق‌شونده تکراری روش جستجوی ناآگاهانه مطلوب می‌باشد.

جستجوی عمیق‌شونده تکراری از این جهت با جستجوی اول‌سطح قابل قیاس است که در هر تکرار، قبل از رفتن به لایه بعدی، لایه گره‌های تازه را به طور کامل کاوش می‌کند. به نظر می‌رسد که به‌کارگیری روشی تکراری، مانند جستجوی هزینه‌یکنواخت، تصمیم‌درستی است. زیرا تضمین بهینگی را از الگوریتم جستجوی هزینه‌یکنواخت به ارث می‌برد ولی به آن اندازه حافظه احتیاج ندارد. اساس این کار بدین صورت است که به جای محدودیتهای عمق در حال افزایش، از محدودیتهای هزینه مسیر در حال افزایش استفاده کنیم. الگوریتمی که حاصل می‌شود، جستجوی طولانی‌کننده تکراری نامیده و در تمرین ۳-۱۱ بررسی می‌شود. متأسفانه، مشخص شده است که جستجوی طولانی‌کننده تکراری سربار قابل توجهی را در مقایسه با جستجوی هزینه‌یکنواخت ایجاد می‌کند.

جستجوی دوطرفه

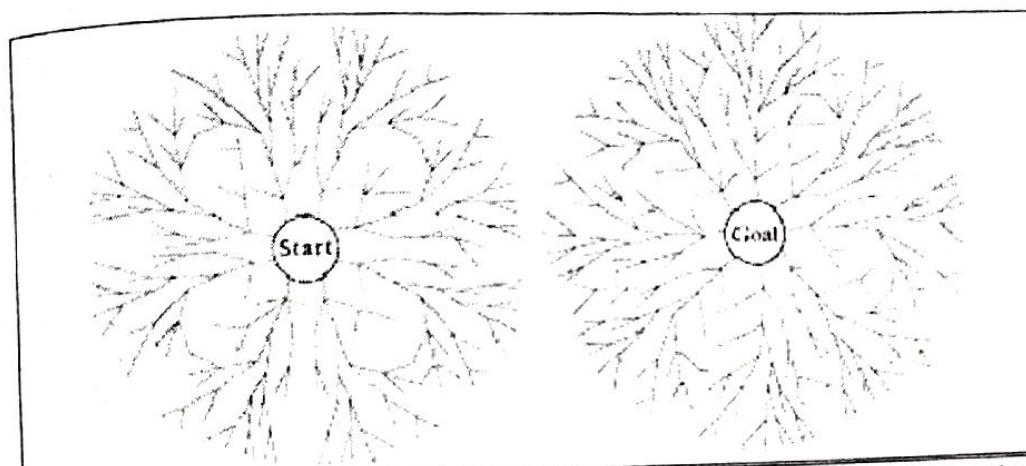
اساس جستجوی دوطرفه انجام دو جستجوی هم‌زمان می‌باشد (یکی از حالت اولیه به سمت جلو و دیگری از هدف به سمت عقب و هنگامی که این دو به هم برسند، متوقف می‌شود (شکل ۳-۱۶)). انگیزه استفاده از این روش این است که $b^{d/2} + b^{d/2}$ بسیار کوچک‌تر از b^d است یا در شکل، سطح دو دایره کوچک، کمتر از سطح دایره بزرگی است که مرکز آن start است و goal را هم شامل می‌شود.

جستجوی دوطرفه به این صورت انجام می‌شود: یکی یا هر دوی جستجوها، قبل از گسترش هر گره بررسی می‌کند که آیا آن گره در لبه درخت جستجوی دیگر قرار دارد یا نه؛ اگر چنین بود، یک راه‌حل پیدا شده است. برای مثال، اگر یک مسأله دارای عمق راه‌حل $d=6$ باشد و جستجوی اول‌سطح در هر جهت به صورت یک گره در هر لحظه انجام شود، در این صورت، در بدترین حالت، دو جستجو زمانی به یکدیگر می‌رسند که هر کدام در تمام گره‌های سطح ۳ به‌جز یکی گسترش پیدا کرده باشند. در مورد $b=10$ این بدان معناست که تعداد کل ۲۲۲۰۰ گره تولید می‌شود که این مقدار در جستجوی اول عرض استاندارد ۱۱۱۱۱۰۰ است. بررسی عضویت یک گره در درخت جستجوی دیگر با یک جدول هش، می‌تواند در زمان ثابتی انجام گیرد. بنابراین، پیچیدگی زمانی جستجوی دوطرفه $O(b^{d/2})$ است. حداقل یکی از درختهای جستجو باید در حافظه نگهداری شود تا بتوان عضویت را بررسی کرد. در نتیجه، پیچیدگی فضایی نیز $O(b^{d/2})$ خواهد بود. نیاز به فضا، بزرگ‌ترین نقطه ضعف جستجوی دوطرفه است. اگر هر دو جستجو از نوع اول عمیق باشند، الگوریتم کامل و بهینه (به خاطر هزینه‌های مراحل

در سطح

یکنواخت) خواهد بود؛ ترکیبات دیگر ممکن است باعث قربانی شدن کامل بودن یا بهینگی یا هر دو شوند.

کاهش پیچیدگی زمانی، باعث جذابیت جستجوی دوطرفه شده است. اما چگونه می‌توان رو به عقب جستجو کرد؟ این کار به آن راحتی که به نظر می‌رسد نیست. فرض کنید پیشینه‌های n (یعنی $Pred(n)$) تمامی آن گره‌هایی باشند که گره n پسین آنهاست. جستجوی دوطرفه نیازمند آن است که $Pred(n)$ به طور مؤثری قابل محاسبه باشد. ساده‌ترین حالت، وقتی است که تمامی اقدامات در فضای حالت، معکوس‌پذیر باشند، تا این که $Pred(n) = Succ(n)$ برقرار باشد. برای حل حالت‌های دیگر به ابتکار نیاز داریم.



شکل ۳-۱۶ یک نمایش طرح‌واره از جستجوی دوطرفه که به موفقیت نزدیک است زیرا هنگامی به موفقیت می‌رسد که یک شاخه از گره $start$ به یک شاخه از گره $goal$ برسد.

در عبارت "جستجوی رو به عقب از هدف"، منظور ما از "هدف" چیست. در پازل هشت‌تایی و در جستجوی راه در رومانی، تنها یک حالت هدف وجود دارد. بنابراین، جستجوی رو به عقب بسیار شبیه به جستجوی رو به جلو است. اگر چندین حالت هدف وجود داشته باشد که صریحاً فهرست شده باشند (برای مثال دو حالت هدف بدون کثیفی در شکل ۳-۳)، در آن صورت می‌توانیم یک حالت هدف ساختگی جدید بسازیم که پیشینه‌های بلافاصله آن، حالت‌های هدف واقعی هستند. با در نظر گرفتن مجموعه حالت‌های هدف به عنوان یک حالت که پیشینه‌های هر یک نیز یک مجموعه حالت هستند، به خصوص مجموعه حالت‌هایی که پسینه‌های متناظری در مجموعه حالت‌های هدف دارند، می‌توان از تولید گره اضافی تا حدودی اجتناب کرد. (بخش ۳-۶ را نیز ملاحظه کنید).

مشکل‌ترین حالت برای جستجوی دوطرفه، هنگامی است که آزمون هدف تنها یک توصیف ضمنی از مجموعه احتمالاً بزرگی از حالت‌های هدف را ارائه می‌کند (برای مثال، تمامی حالت‌هایی که در آزمون هدف کیش و مات در بازی شطرنج صدق می‌کنند). یک جستجوی رو به عقب به ساخت توصیف فشرده‌ای از تمام "حالت‌هایی که با حرکت m_1 به کیش و مات منجر می‌شوند" و ... نیاز دارد و این توصیف‌ها باید در مورد حالت‌هایی که توسط جستجوی رو به جلو تولید شده‌اند مورد آزمون قرار گیرند. هیچ راه کلی برای انجام این کار به صورت مؤثر وجود

$E \Rightarrow$ ف	
Complete	کامل
Optimal	بهینه
Combination	ترکیب
To sacrifice	قربانی کردن
Reduction	کاهش
Backwards	رو به عقب
Predecessor	پیشین
Effectively	به طور مؤثر
Computable	قابل محاسبه
Reversible	معکوس‌پذیر
Creativity	خلاقیت
Schematic	طرح‌واره
8-Puzzle	پازل هشت‌تایی
Explicitly	صریحاً
List	فهرست
Dummy goal	هدف ساختگی
Actual	واقعی
Implicit	توصیف ضمنی
Ingenuity	ابتکار

ندارد.

ف $\Rightarrow$ E

Bidirectional

دوطرفه

Iterative

deepening

عمیق شونده

تکراری

Depth-limited

با عمق محدود

Depth-first

اول عمق

Uniform-cost

هزینه یکنواخت

Breadth-first

اول سطح

Criterion

معیار

Evaluation

ارزیابی

To avoiding

اجتناب کردن

Repeated

تکراری

To waste

به هدر دادن

Formulation

تدوین

Accessible

قابل دسترسی

Unavoidable

غیرقابل اجتناب

Reversible

معکوس پذیر

Route-finding

مسیریابی

Sliding -

blocks

بلاکهای لغزنده

Finite

متناهی

Comparison

مقایسه

مقایسه راهبردهای جستجوی ناآگاهانه

شکل ۳-۱۷ راهبردهای جستجو را بر اساس چهار معیار ارزیابی که در بخش ۳-۴ مطرح گردید با هم مقایسه می کند.

معیار	دوطرفه (در صورت امکان)	عمیق شونده تکراری	با عمق محدود	اول عمق	هزینه یکنواخت	اول سطح
کامل؟	Yes ^{a,d}	Yes ^a	No	No	Yes ^{a,b}	Yes ^a
زمان	$O(b^{d/2})$	$O(b^d)$	$O(b^l)$	$O(b^m)$	$O(b^{l+c+l})$	$O(b^{d+l})$
فضا	$O(b^{d/2})$	$O(bd)$	$O(bl)$	$O(bm)$	$O(b^{l+c+l})$	$O(b^{d+l})$
بینه؟	Yes ^{c,d}	Yes ^c	No	No	Yes	Yes ^c

شکل ۳-۱۷ ارزیابی راهبردهای جستجو. b ضریب انشعاب، d عمق کم عمق ترین راه حل، m حداکثر عمق درخت جستجو و l حد عمق می باشد. بالانویسها به شرح زیر هستند: "کامل به شرطی که b متناهی باشد؛ "کامل به شرطی که هزینه مرحله از $\in$ بزرگتر باشد ($\in$ مثبت است)؛ "بینه در صورتی که هزینه های مراحل همگی یکسان باشند؛ "اگر در هر دو جهت از جستجوی اول سطح استفاده شود.

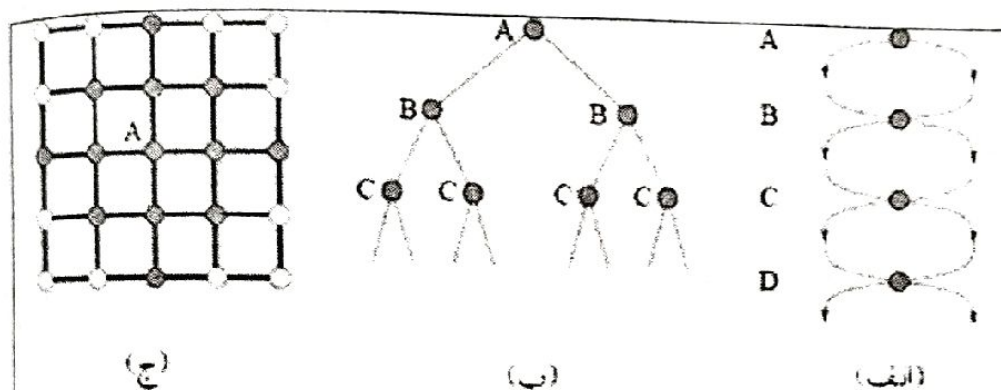
۳-۵ اجتناب از حالت های تکراری

تا این مرحله، یکی از مهم ترین دشواری های فرآیند جستجو را نادیده گرفته ایم: امکان به هدر دادن زمان به خاطر گسترش حالت هایی که قبلاً با آنها مواجه شده ایم و گسترششان نیز داده ایم. در بعضی مسائل، این امکان هرگز به وجود نمی آید؛ فضای حالت به شکل یک درخت می باشد و برای هر حالت تنها یک مسیر وجود دارد. تدوین مؤثر مسأله ۸ وزیر (که هر وزیر جدید در ستون خالی که در منتهی علیه سمت چپ قرار دارد، جای می گیرد) بیشتر به همین علت مؤثر می باشد، زیرا هر حالت تنها از یک طریق قابل دست یابی است. اگر مسأله ۸ وزیر را به این صورت تدوین کنیم که یک وزیر در هر ستونی بتواند قرار گیرد، در آن صورت هر حالت با n وزیر، با استفاده از $n!$ راه مختلف قابل دسترسی خواهد بود.

در مورد بعضی مسائل، حالت های تکراری غیرقابل اجتناب هستند. این موضوع تمام مسائلی را که دارای اقدامات معکوس پذیر هستند، مانند مسائل مسیریابی و مسائل بلاکهای لغزنده، شامل می شود. درخت های جستجوی این مسائل، نامتناهی هستند. اما در صورتی که برخی از حالت های تکراری را حذف کنیم، می توانیم درخت جستجو را به اندازه متناهی برسانیم و فقط آن قسمتی از درخت را تولید کنیم که گراف فضای حالت را گسترش می دهد. با در نظر گرفتن درخت جستجو تنها تا یک عمق ثابت، یافتن حالت هایی که در آنها، حذف حالت های تکراری باعث کاهش نمایی هزینه جستجو می شود، ساده است. در شدیدترین حالت، یک فضای حالت با اندازه $d+1$

(شکل ۳-۱۸ الف)) درختی با 2^d برگ خواهد شد (شکل ۳-۱۸ ب)). یک مثال واقعیت، شبکه مستطیلی است که در شکل ۳-۱۸ ج) نشان داده شده است. در یک شبکه، هر حالت چهار پسین دارد. بنابراین، درخت جستجو که شامل حالت‌های تکراری نیز می‌باشد، دارای 4^d برگ است؛ در حالی که تنها در حدود $2d^2$ حالت متمایز در d مرحله از هر حالت خاص وجود دارد. در مورد $d=20$ ، حدود یک تریلیون گره با تنها حدود ۸۰۰ حالت متمایز وجود دارد.

اگر حالت‌های تکراری توسط الگوریتم شناسایی نشود، این حالت‌های تکراری می‌توانند یک مسئله قابل حل را به یک مسئله غیر قابل حل تبدیل کنند. معمولاً منظور از شناسایی، مقایسه گره در شرف گسترش با گره‌هایی که پیش از این گسترش یافته‌اند می‌باشد. اگر تطابقی پیدا شود آنگاه الگوریتم دو مسیر را که به یک حالت می‌رسند کاوش کرده است و می‌تواند یکی از آنها را دور بیندازد.



شکل ۳-۱۸ فضاهای حالتی که درخت جستجوی به طور نمایی بزرگتری را تولید می‌کنند. (الف) فضای حالتی که در آن دو اقدام ممکن از A به B ، دو اقدام از B به C و غیره وجود دارد. فضای حالت دارای $d+1$ حالت است که d حداکثر عمق است. (ب) درخت جستجوی متناظر که 2^d شاخه متناظر با 2^d مسیر درون فضا دارد. (ج) یک فضای شبکه‌ای مستطیل شکل. حالت‌هایی که در فاصله دو مرحله‌ای حالت اولیه (A) قرار دارند، به رنگ خاکستری نشان داده شده‌اند.

در جستجوی اول عمق، تنها گره‌هایی در حافظه نگهداری می‌شوند که روی مسیر فعلی از ریشه تا گره قرار دارند. مقایسه این گره‌ها با گره فعلی به الگوریتم امکان شناسایی مسیرهای حلقه‌ای را می‌دهد که می‌توانند بلافاصله کنار گذاشته شوند. این موضوع از آن جهت با ارزش است که از تبدیل فضاهای حالت متناهی به درختان جستجوی نامتناهی به علت وجود حلقه‌ها جلوگیری می‌کند، ولی متأسفانه از ازدیاد نمایی مسیرهای غیر حلقوی در مسائلی مانند شکل ۳-۱۸، نمی‌تواند پیش‌گیری کند. تنها راه اجتناب از به وجود آمدن این مسیرها، نگهداری گره‌های بیشتری در حافظه است. یک تناسب همیشگی بین فضا و زمان وجود دارد: الگوریتم‌هایی که گذشته خود را فراموش کنند، محتوم به تکرار آن هستند.

اگر الگوریتمی تمام حالت‌هایی که از آنها گذشته است را به خاطر داشته باشد، آنگاه می‌توان گفت که این الگوریتم گراف فضای حالت را مستقیماً کاوش می‌کند. می‌توان الگوریتم عمومی $TREE-SEARCH$ را به گونه‌ای تغییر داد که شامل ساختار داده‌ای به نام فهرست

ف E	
Rectangular grid	شبکه مستطیلی
Leaf	برگ
Distinct	متمايز
To detect	شناسایی کردن
Unsolvable	غير قابل حل
To discard	دور انداختن
Exponentially	به طور نمایی
Grid space	فضای شبکه‌ای
Gray	خاکستری
Comparison	مقایسه
Current node	گره فعلی
Looping paths	مسیرهای حلقه‌ای
For ensuring	برای اطمینان از
To convert	تبدیل کردن
Exponential proliferation	ازدیاد نمایی
Tradeoff	مصلحه
Fundamental	بنیادی
To explore	کاوش کردن

سوال ۱: شرح graphsearch و tree search چیست؟

۹۹

فصل ۳- حل مسأله از طریق جستجو

ف E	گره‌های گسترش یافته
Expanded nodes	گسترش یافته
گسترش یافته	لبه
Fringe	To match
لبه	مطابقت داشتن
To match	Proportional
مطابقت داشتن	متناسب
Proportional	Optimality
متناسب	بهینگی
Optimality	Short
بهینگی	کوتاه
Short	Fortunately
کوتاه	خوشبختانه
Fortunately	Suboptimal
خوشبختانه	نیمه بهینه
Suboptimal	Better
نیمه بهینه	بهرتر
Better	Depth
بهرتر	عمق
Depth	Cost
عمق	هزینه
Cost	General
هزینه	عمومی
General	Cheap
عمومی	ارزان
Cheap	List
ارزان	فهرست
List	Linear space
فهرست	فضای خطی
Linear space	Memory limitation
فضای خطی	محدودیت
Memory limitation	حافظه
محدودیت	Feasible
حافظه	عملی
Feasible	Repeated
عملی	تکراری
Repeated	
تکراری	

closed شود که تمام گره‌های گسترش یافته را ذخیره می‌کند. (لبه گره‌های گسترش نیافته را گاهی **فهرست open** می‌نامند) اگر گره فعلی با یکی از گره‌های **فهرست closed** مطابقت داشته باشد به جای این که گسترش یابد، کنار گذاشته می‌شود. این الگوریتم جدید **GRAPH-SEARCH** نامیده می‌شود (شکل ۳-۱۹). در مسائلی که حالت‌های تکراری زیادی دارند، **GRAPH-SEARCH** بسیار مؤثرتر از **TREE-SEARCH** عمل می‌کند. زمان و فضای مورد نیاز آن در بدترین حالت، متناسب با اندازه فضای حالت است و ممکن است بسیار کمتر از $O(b^d)$ باشد.

function GRAPH-SEARCH(problem, fringe) returns a solution, or failure

```

closed ← an empty set
fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
loop do
  if EMPTY?(fringe) then return failure
  node ← REMOVE-FIRST(fringe)
  if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
  if STATE[node] is not in closed then
    add STATE[node] to closed
    fringe ← INSERT-ALL(EXPAND(node, problem), fringe)

```

شکل ۳-۱۹ الگوریتم عمومی جستجوی گراف. مجموعه **closed** توسط یک جدول هش قابل پیاده‌سازی است تا بررسی حالت‌های تکراری به طور مؤثر انجام شود. این الگوریتم فرض می‌کند که اولین مسیر به حالت s ارزان‌ترین است (به متن رجوع کنید).

شکل ۳-۱۹ الگوریتم عمومی جستجوی گراف. مسئله بغرنجی است. پیش از این بیان کردیم که وقتی که یک حالت تکراری شناسایی شد، الگوریتم دو مسیر برای یک حالت یافته است. الگوریتم **GRAPH-SEARCH** در شکل ۳-۱۹ همیشه مسیر جدیدی که کاوش شده است را دور می‌اندازد. واضح است اگر مسیر جدیدی که پیدا شده است کوتاه‌تر از مسیر اصلی باشد، ممکن است **GRAPH-SEARCH** یک جواب بهینه را از دست داده باشد. خوشبختانه می‌توان نشان داد (تمرین ۳-۱۲) وقتی که از جستجوی هزینه یکنواخت یا جستجوی اول سطح با هزینه مرحله ثابت استفاده می‌کنیم، این اتفاق نمی‌افتد. در نتیجه، این دو راهبرد جستجوی بهینه درخت، راهبرد جستجوی بهینه گراف نیز هستند. از سوی دیگر، جستجوی عمیق‌شونده تکراری از گسترش اول عمق استفاده می‌کند و ممکن است به سادگی، پیش از یافتن مسیر بهینه، مسیر نیمه بهینه‌ای را به یک گره دنبال کند. در نتیجه، گراف عمیق‌شونده تکراری باید بررسی کند که آیا مسیر تازه پیدا شده به یک گره، از مسیر اصلی بهتر است و اگر چنین بود، ممکن است لازم باشد عمق و هزینه مسیر پیشین‌های آن گره را بازمینی کند.

توجه داشته باشید که استفاده از **فهرست closed** به این معناست که جستجوی اول عمق و جستجوی عمیق‌شونده تکراری، دیگر به فضای خطی نیازی ندارند. از آنجا که الگوریتم **GRAPH-SEARCH** تمام گره‌ها را در حافظه نگه می‌دارد، انجام بعضی از جستجوها به دلیل محدودیت حافظه، عملی نیستند.

سوال، پیچیدگی حافظه‌ای جستجوی اول عمق در جستجوی فضایی بهینه است یا جستجوی گراف؟

۶-۳ جستجو با اطلاعات ناقص

در بخش ۳-۳ فرض کردیم که محیط کاملاً رؤیت‌پذیر و معین است و کارگزار می‌داند که اثر هر اقدام چیست. در نتیجه، کارگزار می‌تواند دقیقاً محاسبه کند که انجام یک رشته اقدام، به کدام حالت منجر می‌شود و در هر لحظه می‌داند که در چه حالتی قرار دارد. پس از انجام یک اقدام، ادراکات آن کارگزار، هیچ اطلاعات جدیدی ارائه نمی‌کنند. اگر دانش حالتها یا اقدامات ناقص باشد، چه اتفاقی می‌افتد؟ انواع مختلف نقص (کامل نبودن)، به سه نوع مسأله مختلف منجر می‌شوند:

- (۱) **مسائل بدون حس‌گر** (که مسائل انطباقی نیز نامیده می‌شوند): اگر کارگزار هیچ حس‌گری نداشته باشد، آنگاه (تا آنجا که می‌داند) ممکن است در یکی از چند حالت اولیه ممکن قرار داشته باشد و در نتیجه، انجام یک اقدام، ممکن است به یکی از چند حالت پسین ممکن، منجر شود.
- (۲) **مسائل اقتضایی**: اگر محیط نیمه‌رؤیت‌پذیر باشد یا اقدامات غیرقطعی باشند، آنگاه پس از انجام هر اقدام، ادراکات کارگزار، اطلاعات جدیدی فراهم می‌کنند. هر ادراک ممکن، اقتضایی را تعریف می‌کند که باید برای آن برنامه‌ریزی کرد. اگر این عدم قطعیت ناشی از اقدامات یک کارگزار دیگر باشد، آن مسأله را یک مسأله تخصصی می‌نامیم.
- (۳) **مسائل کاوشی**: هنگامی که حالتها و اقدامات محیط ناشناخته است، کارگزار باید برای کاوش آنها اقدام کند. مسائل کاوشی را می‌توان حد نهایی مسائل اقتضایی دانست. در این بخش، از محیط دنیای جاروبرقی به عنوان نمونه استفاده می‌کنیم. به خاطر می‌آورید که فضای حالت این دنیا شامل هشت حالت است که در شکل ۳-۲۰ نشان داده شده است. همچنین سه اقدام وجود دارد (*Right*, *Left* و *Suck*) و هدف تمیز کردن تمام آشغالهاست (حالت‌های ۷ و ۸). اگر محیط رؤیت‌پذیر، معین و کاملاً شناخته‌شده باشد، آنگاه این مسأله به کمک هر یک از الگوریتم‌هایی که تاکنون توضیح داده‌ایم به سادگی قابل حل است. برای مثال، اگر حالت اولیه ۵ باشد، آنگاه رشته اقدامات [*Right*, *Suck*] به حالت هدف ۸ خواهد رسید. باقیمانده این بخش درباره انواع بدون حس‌گر و اقتضایی این مسأله است. به مسائل کاوشی در بخش ۴-۵ و به مسائل تخصصی در فصل ۶ پرداخته می‌شود.

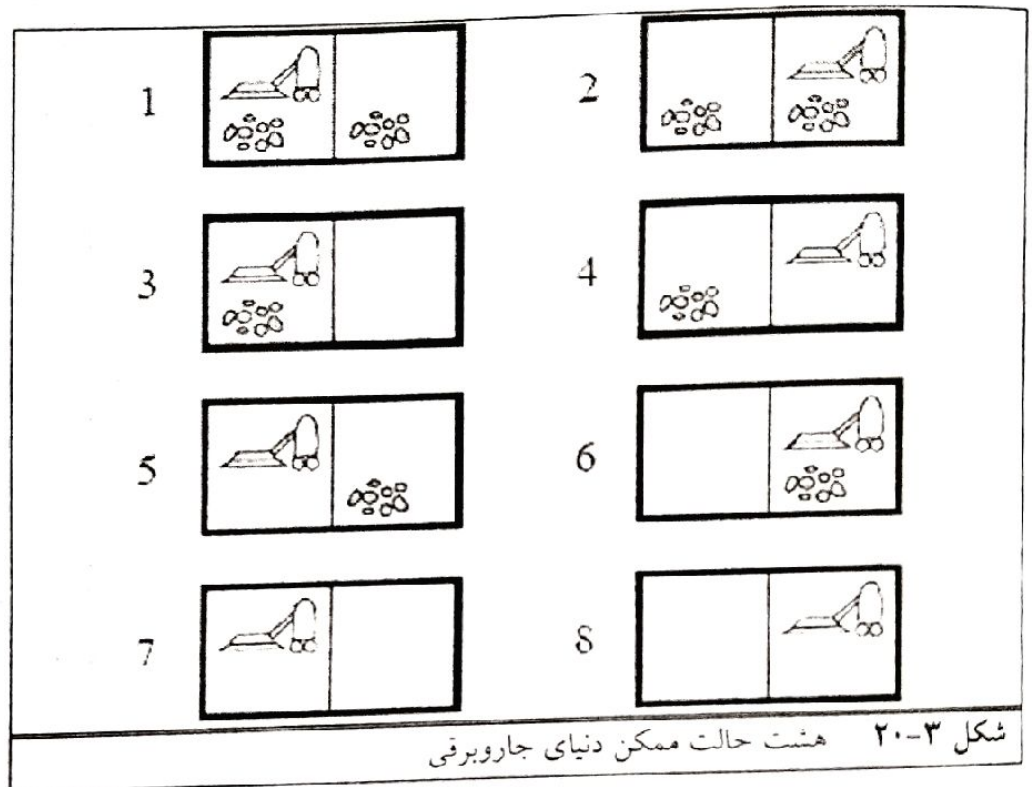
مسائل بدون حس‌گر

فرض کنید که کارگزار جاروبرقی از تمامی تأثیرات اقداماتش آگاه است ولی هیچ حس‌گری ندارد. در این صورت، تنها می‌داند که حالت اولیه‌اش یکی از اعضای مجموعه {۸ و ۷ و ۶ و ۵} و ۴ و ۳ و ۲ و ۱ است. ممکن است فکر کنید که وضعیت این کارگزار ناامیدکننده است، اما واقعیت این است که می‌تواند نسبتاً خوب عمل کند. از آنجا که می‌داند که اقداماتش چه کاری انجام می‌دهند، می‌تواند برای مثال، محاسبه کند که اقدام *Right* باعث می‌شود که در یکی از حالت‌های {۸ و ۶ و ۴ و ۲} قرار گیرد و رشته اقدامات [*Right*, *Suck*] همیشه باعث می‌شود که در یکی از حالت‌های {۸ و ۴} قرار بگیرد. بالاخره رشته [*Right*, *Suck*, *Left*, *Suck*] بدون

ف E	Searching with partial information
جستجو با اطلاعات ناقص	Fully observable
کاملاً رؤیت‌پذیر	Deterministic
معین	Effect
اثر	To lead to
منجر شدن	Percepts
ادراکات	Knowledge
دانش	Incomplete
ناقص	Sensorless problems
مسائل بدون حس‌گر	Conformant problems
مسائل انطباقی	Contingency
اقتضایی	Adversarial
تخصصی	Exploration
کاوش	Successor
پسین	Partially observable
نیمه‌رؤیت‌پذیر	Uncertain
غیرقطعی	To plan
برنامه‌ریزی کردن	

ف	E
Deterministic	معین
Completely known	کاملاً شناخته شده
Solvable	قابل حل
Sensorless	بدون حس گر
Contingency	اقتضائی
Initial state	حالت اولیه
Hopeless	نامید کننده
To guarantee	تضمین کردن
To coerce	مجبور کردن
To reason	استدلال کردن
Belief state	حالت باور
Possible physical states	حالت های فیزیکی
Actual	ممکن
To apply	واقعی
Union	اجتماع
Results	نتایج
Non-deterministic	نامعین
To map	نگاشتن

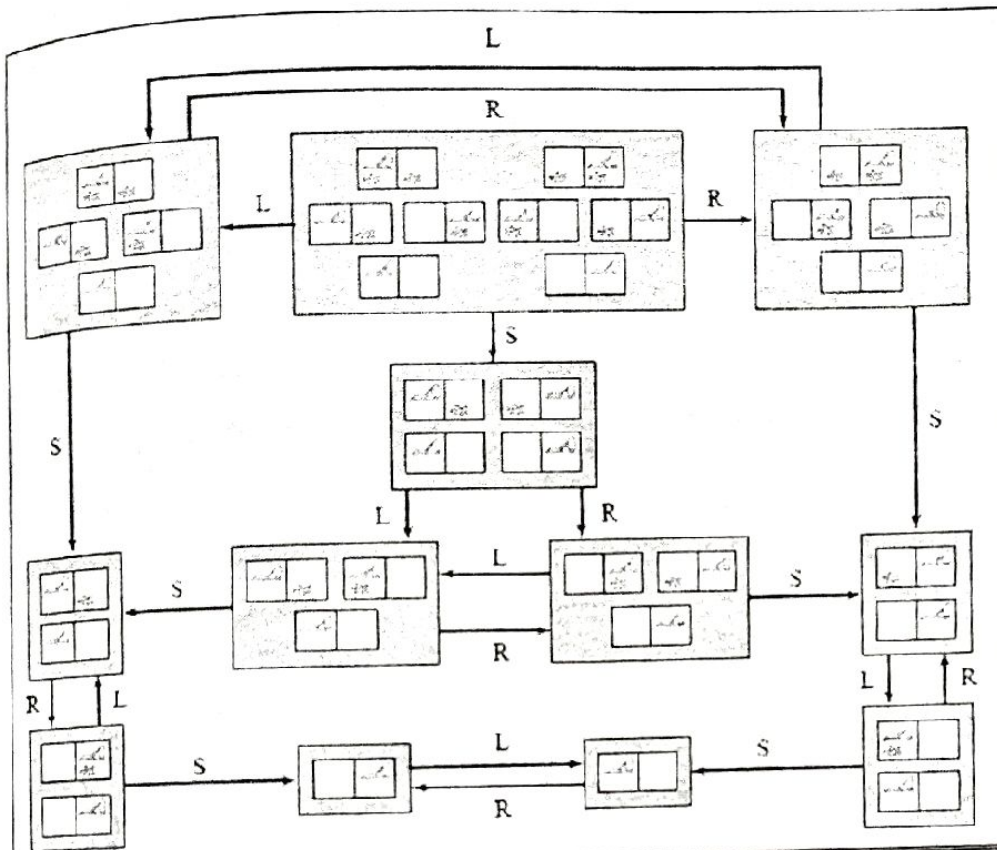
توجه به حالت اولیه تضمین می کند که به حالت هدف ۷ می رسیم. بدین ترتیب می توان گفت که کارگزار می تواند دنیا را مجبور کند به حالت ۷ برود، حتی هنگامی که نمی داند که از کجا شروع کرده است. خلاصه مطلب چنین شد: در زمانی که دنیا کاملاً رؤیت پذیر نیست، کارگزار باید به جای یک حالت، درباره مجموعه حالت هایی که ممکن است در آنها قرار گیرد استدلال کند. به چنین مجموعه حالتی، حالت باور می گوئیم که باور فعلی کارگزار را درباره حالت های فیزیکی ممکن که ممکن است در آنها قرار داشته باشد نشان می دهد (در یک محیط کاملاً رؤیت پذیر، هر حالت باور شامل یک حالت واقعی است).



برای حل مسائل بدون حس گر، به جای فضای حالت های فیزیکی، در فضای حالت های باور جستجو می کنیم. حالت اولیه یک حالت باور است و هر اقدام یک حالت باور را به یک حالت باور دیگر ارتباط می دهد. اعمال یک اقدام به یک حالت باور، با اجتماع نتایج اعمال آن اقدام بر تک تک حالت های واقعی درون آن حالت باور انجام می گیرد. در اینجا یک مسیر چندین حالت باور را به هم وصل می کند و راه حل مسیری است که به فضای باوری منجر می شود که تمامی اعضایش حالت هدف هستند. شکل ۳-۲۱ فضای باور دسترس پذیر را برای دنیای معین بدون حس گر جاروبرقی نشان می دهد. تنها ۱۲ حالت باور دسترس پذیر وجود دارد، اما کل فضای حالت باور، تمامی مجموعه حالت های واقعی ممکن یعنی $2^8 = 256$ حالت باور را شامل می شود. در حالت کلی، اگر فضای حالت واقعی دارای k حالت باشد، فضای حالت باور، شامل 2^k حالت باور خواهد بود.

در مباحثی که تا اینجا درباره مسائل بدون حس گر مطرح شد فرض بر این بود که اقدامات معین هستند، اما اگر محیط نامعین باشد (یعنی اقدامات دارای نتایج متعددی باشند)،

تغییر اساسی در تحلیل ما رخ نمی‌دهد. علت این امر آن است که در نبود حس‌گرها، کارگزار هیچ راهی برای تعیین نتیجه واقعی ندارد. بنابراین پیش‌آمدهای مختلف ممکن تنها چند حالت فیزیکی اضافی در حالت باور پسین هستند.



شکل ۳-۲۱ بخش قابل دسترسی فضای حالت باور در دنیای معین بدون حس‌گر جاروبرقی. هر مستطیل خاکستری متناظر با یک حالت باور است. در هر نقطه، کارگزار در یک حالت باور قرار دارد ولی نمی‌داند که در کدام حالت واقعی است. حالت باور اولیه (نادانی کامل) مستطیل وسطی در بالای شکل است. اقدامات توسط پیکانهای برچسب‌دار نشان داده شده‌اند. به منظور وضوح و خوانایی، حلقه‌هایی که به خود حالت برمی‌گردند حذف شده‌اند.

برای مثال فرض کنید این محیط از قانون مورفی تبعیت می‌کند: اقدامی که به اصطلاح مکش (Suck) نامیده می‌شود گاهی اوقات آشغالها را روی فرش خالی می‌کند، البته فقط هنگامی که هیچ آشغالی در آن لحظه آنجا نباشد.^۶ در این صورت، اگر مکش به حالت واقعی ۴ اعمال شود دو نتیجه ممکن وجود دارد: حالت‌های ۲ و ۴ (شکل ۳-۲۰ را نگاه کنید). با اعمال مکش به حالت باور اولیه، یعنی ۱، ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱، به حالت باوری می‌رسیم که اجتماع مجموعه پیش‌آمدهای هشت حالت واقعی است. با انجام این محاسبه درمی‌یابیم که

ف $\Rightarrow$ E

Successor belief state

حالت باور پسین

Reachable portion

بخش قابل

دسترس

The deterministic, sensorless vacuum world

دنیای معین

بدون حس‌گر

جاروبرقی

Shaded box

مستطیل

خاکستری

Clarity

وضوح

To omit

حذف کردن

Loop

حلقه

Murphy's law

قانون مورفی

Calculation

محاسبه

Union

اجتماع

To obey

اطاعت کردن

Dirt

آشغال

Complete ignorance

نادانی کامل

To suck

مکیدن

^۶ فرض می‌کنیم که اکثر خوانندگان با مشکلات مشابهی روبرو هستند و می‌توانند با کارگزار ما ارتباط برقرار کنند. از کسانی که دارای دستگاههای جاروبرقی مدرن و پیشرفته هستند و نمی‌توانند از این مثال بهره‌مند شوند پوزش می‌طلبیم.

E	ف
To solve	حل کردن
Contingency	اقتضایی
Position sensor	حس گر موقعیت
Local dirt sensor	حس گر محلی
Action sequence	آشغال
To formulate	رشته اقدام
Suck	تدوین کردن
Goal	مکش
To fail	هدف
Belief-state space	با شکست مواجه شدن
Version	فضای حالت باور
Fixed	نسخه
To extend	ثابت
Prediction	گسترش دادن
Exact	پیش بینی
Sequential solutions	دقیق
	راه حل های متوالی

حالت باور جدید {۸ و ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱} می باشد. بنابراین، برای یک کارگزار بدون حس گر در دنیای قانون مورفی، اقدام مکش حالت باور را تغییر نمی دهد! در واقع، مسأله غیر قابل حل است. (تمرین ۳-۱۸ را نگاه کنید.) می توان گفت علت این امر آن است که کارگزار نمی تواند تشخیص دهد مربع فعلی کثیف است و در نتیجه نمی تواند تشخیص دهد که اقدام مکش آن را تمیز می کند یا کثیفی بیشتری به وجود می آورد.

مسائل اقتضایی

هنگامی که محیط به گونه ای است که کارگزار می تواند پس از اقدام، اطلاعات جدیدی را از طریق حس گرهایش به دست آورد، کارگزار با یک مسأله اقتضایی روبروست. راه حل یک مسأله اقتضایی اغلب به شکل یک درخت است که هر شاخه آن درخت ممکن است بر اساس ادراکاتی که تا آن نقطه درخت دریافت شده است، انتخاب شود. برای مثال، فرض کنید که کارگزار در دنیای قانون مورفی قرار دارد و دارای یک حس گر موقعیت و یک حس گر محلی آشغال می باشد ولی فاقد حس گری است که قادر به شناسایی آشغال در مربعهای دیگر باشد. بنابراین، ادراک $[L, Dirty]$ به این معنی است که کارگزار در یکی از حالت های {۱,۳} قرار دارد. کارگزار ممکن است رشته اقدام $[Suck, Right, Suck]$ را تدوین کند. مکش حالت را به یکی از {۵,۷} تغییر می دهد و سپس حرکت به راست حالت را به یکی از {۶,۸} تغییر می دهد. انجام اقدام نهایی مکش در حالت ۶ ما را به حالت ۸ که هدف می باشد می برد، اما اگر در حالت ۸ این اقدام اجرا شود، ممکن است ما را دوباره به حالت ۶ برگرداند (بر اساس قانون مورفی) که در این صورت برنامه با شکست مواجه می شود.

با بررسی فضای حالت باور در مورد این نسخه از مسأله، به سادگی معلوم می شود که هیچ رشته اقدام ثابتی وجود ندارد که یافتن راه حل این مسأله را تضمین کند. با وجود این، اگر بر یک رشته اقدام ثابت اصرار نداشته باشیم، یک راه حل وجود دارد:

$[Suck, Right, if [R, Dirty] then Suck]$

بدین طریق، فضای راه حلها گسترش می یابد تا امکان انتخاب اقدامات بر اساس اقتضائاتی که در حین اجرا پیش می آید را نیز شامل شود. از آنجا که پیش بینی دقیق بسیاری از مسائل در دنیای واقعی غیر ممکن است این گونه مسائل، مسائلی اقتضایی محسوب می شوند. به همین دلیل است که اکثر مردم در هنگام راه رفتن یا رانندگی چشمانشان را باز نگه می دارند.

گاهی اوقات در مسائل اقتضایی راه حل های صرفاً متوالی امکان پذیر می باشد. برای مثال، یک دنیای قانون مورفی کاملاً رؤیت پذیر را در نظر بگیرید. اقتضائات در صورتی پیش می آیند که کارگزار به دلیل آنکه ممکن است آشغال موجود در مربع تمیز تخلیه شود یا ممکن است تخلیه نشود، اقدام مکش را در آن مربع انجام می دهد. تا زمانی که کارگزار چنین کاری را انجام ندهد، هیچ اقتضایی پیش نمی آید و یک راه حل متوالی از حالت اولیه وجود خواهد داشت (تمرین ۳-۱۸).

الگوریتم های مسائل اقتضایی، پیچیده تر از الگوریتم های جستجوی استاندارد است که در این فصل آورده شده اند. به این الگوریتمها در فصل ۱۲ می پردازیم. مسائل اقتضایی همچنین می تواند ناشی از کارگزارهایی با طرح های متفاوت باشد که در آن کارگزار پیش از یافتن یک

برنامه ریزی تضمینی. اقدام می‌کند. این موضوع بسیار سودمند است زیرا به جای بررسی پیشاپیش تمامی اقتضائاتی که ممکن است در حین اجرا اتفاق بیافتد، اغلب بهتر است شروع به اقدام نماییم و ببینیم که چه اقتضائاتی واقعاً پیش می‌آیند. آنگاه کارگزار می‌تواند با در نظر گرفتن اطلاعات اضافی، به حل مسأله ادامه دهد. این نوع درآمیختگی اطلاعات جستجو و اجرا در مسائل کاوشی (بخش ۴-۵ را ببینید) و انجام بازی (فصل ۶ را ببینید) سودمند هستند.

۷-۳ خلاصه

در این فصل، روشهایی معرفی گردید که یک کارگزار می‌تواند در انتخاب اقدامات برای محیطهای معین، رؤیت‌پذیر، ایستا و کاملاً شناخته‌شده، از آنها استفاده کند. در چنین مواردی، کارگزار می‌تواند رشته اقداماتی تولید کند که آن را به هدف برسانند. این فرآیند را جستجو می‌نامند.

- پیش از آن که یک کارگزار جستجوی راه‌حل را آغاز کند، باید یک هدف را تدوین و آنگاه از این هدف برای تدوین مسأله استفاده کند.

- هر مسأله از چهار جزء تشکیل می‌شود: حالت اولیه، یک مجموعه از اقدامات، یک تابع آزمون هدف و یک تابع هزینه مسیر. محیط مسأله توسط یک فضای حالت بیان می‌شود. یک مسیر از حالت اولیه در درون فضای حالت به یک حالت هدف، راه‌حل نامیده می‌شود.

- الگوریتم ساده و عمومی *TREE-SEARCH* می‌تواند برای حل هر مسأله‌ای استفاده شود. گونه‌های مختلف این الگوریتم، راهبردهای مختلف را تشکیل می‌دهند.

- الگوریتمهای جستجو بر اساس کامل بودن، بهینگی، پیچیدگی زمانی و پیچیدگی فضایی بررسی می‌شوند. پیچیدگی به ضریب انشعاب در فضای حالت (یعنی b) و عمق کم عمق‌ترین راه‌حل (یعنی d) بستگی دارد.

- جستجوی اول سطح کم عمق‌ترین گره گسترش نیافته در درخت جستجو را برای گسترش انتخاب می‌کند. این روش کامل است، با فرض هزینه ثابت واحد برای هر مرحله، بهینه است و پیچیدگی زمانی و فضایی $O(b^d)$ دارد. این پیچیدگی فضایی باعث می‌شود که این روش در اکثر موارد غیر عملی شود. جستجوی هزینه یکنواخت مشابه جستجوی اول سطح می‌باشد ولی گرهی را که کمترین هزینه مسیر (یعنی $g(n)$) را داراست، گسترش می‌دهد. اگر هزینه تک تک مراحل از یک حد مثبت، مانند ϵ بیشتر باشد، کامل و بهینه است.

- جستجوی اول عمق عمیق‌ترین گره گسترش نیافته درخت را برای گسترش انتخاب می‌کند. این روش نه کامل و نه بهینه است و دارای پیچیدگی زمانی $O(b^m)$ و پیچیدگی فضایی $O(bm)$ می‌باشد که m حداکثر عمق تمامی مسیرها در فضای حالت است.

- جستجوی با عمق محدود، به جستجوی اول عمق، یک محدودیت ثابت برای عمق تحمیل می‌کند.

- جستجوی عمیق‌شونده تکراری، جستجوی با عمق محدود را با محدودیتهای در

ف E

Additional information

اطلاعات اضافی

Exploration problems

مسائل کاوشی

Game playing

انجام بازی

Interleaving

درآمیختگی

Deterministic

معین

Observable

رؤیت‌پذیر

Static

ایستا

Completely known

کاملاً

شناخته‌شده

Process

فرآیند

Formulation

تدوین

Part

جزء

Initial state

حالت اولیه

Action

اقدام

Goal test

آزمون هدف

Path cost function

تابع هزینه مسیر

Optimality

بهینگی

$E \Rightarrow$	حال افزایش فراخوانی می‌کند تا زمانی که یک هدف پیدا شود. این روش کامل است و در صورتی که هزینه گام واحد باشد، بهینه و دارای پیچیدگی زمانی $O(b^d)$ و پیچیدگی فضایی $O(bd)$ است.
Iterative deepening search	جستجوی دوطرفه می‌تواند پیچیدگی زمانی را به طور قابل ملاحظه‌ای کاهش دهد. اما همیشه عملی نیست و ممکن است به فضای بسیار زیادی نیاز داشته باشد.
جستجوی عمیق‌شونده تکراری	زمانی که فضای حالت، به جای درخت، یک گراف است، بررسی درخت جستجو برای یافتن حالت‌های تکراری کار باارزشی است. الگوریتم <i>GRAPH-SEARCH</i> تمامی حالت‌های تکراری را حذف می‌کند.
Increasing در حال افزایش To call	• هنگامی که محیط نیمه‌رؤیت‌پذیر است، کارگزار می‌تواند الگوریتم‌های جستجو را در فضای حالت‌های باور، یا مجموعه حالت‌هایی که کارگزار ممکن است در آنها باشد، انجام دهد. در بعضی موارد یک رشته راه‌حل می‌تواند ساخته شود. در موارد دیگر، کارگزار به یک طرح اقتضایی نیاز دارد تا بتواند از عهده شرایط ناشناخته‌ای که ممکن است پیش آید، برآید.
Bidirectional search	
جستجوی دوطرفه	
Belief states	
حالت‌های باور	
Contingency plan	
طرح اقتضایی	
Bibliography	
کتاب‌شناسی	